IKP – usmeni

I KARAKTERISTIKE MEMORIJSKIH SISTEMA

1. Osnovne karakteristike memorijeskih sistema (polozaj, kapacitet, jedinica prenosa, performance, fizicka realizacija, fizicke karakteristike)
   - ➔ Polozaj memorije:
     - Prema tome da li je memorija unutar ili van racunara podela je na:
       - Unutrasnja memorija: glavna memorija, registry procesora, kes memorija
       - Spoljasnja memorija- su periferijski uredjaji za skladistenje podataka Kojima se pristupa putem U/I kontrolera: opticki diskovi, magnetni diskovi, magnetne trake
   - ➔ Kapacitet memorije
     - Za unutrasnju memoriju tipicno se iskazuje kao broj reci (8, 16, 32 bita) ili bajtova
     - Za spoljasnju memoriju tipicno se iskazuje u bajtovima
   - ➔ Jedinica prenosa
     - Za unutrasnju memoriju moze biti jednaka duzini reci, ali je obicno duza npr. 64, 124 ili 256 bita
     - Jednaka je broju elektricnih veza koje ulaze u memorijski modul ili iz njega izlaze
     - Rec – je "prirodna" jedinica organizacije memorije i obicno je jednaka broju bitova potrebnih za predstavljanje celog broja i duzini instrukcije
       - Intelx86 procesori imaju raznovrsne duzine instrukcija (iskazane u umnoscima bajtova) i ima reci velicine 32 bita
     - Adresibilnajedinica – u nekim sistemima to je rec
       - Neki sistemi dozvoljavaju adresiranje na nivou bajtova
     - Jedinica prenosa
       - Za unutr. mem. = je broju bajtova koji se mogu istovremeno procitati iz memorije ili upisati u memoriju
       - Za spolj. mem. podaci se obicno prenose u vecim jedinicama tzv. blokovi podataka
   - ➔ Performance memorije
     - Vreme pristupa (kasnjenje)
       - Za memoriju sa nasumicnim pristupom to je vreme potrebno da se obavi operacija citanja ili upisivanja
     - Trajanje memorijskog ciklusa
       - Odnosi se na memoriju sa nasumicnim pristupom
       - Vreme pristupa + dodatno vreme kako bi sledeci pristup mogao da pocne (prelazna stanja na sigurnim linijama)
     - Brzina prenosa
       - Brzina kojom e podaci prenesu u/iz memorije
       - Sa nasumicnim pristupom jednako je $\frac{1}{trajanje\ ciklusa}$
   - ➔ Fizicka realizacija memorije
     - Poluprovodnicka

- Magnetna
- Opticka
- Magnetno – opticka

➔ Fizicke karakteristike memorije
- Postojana/nepostojana
  - Nepostojana: podaci se gube kad se iskljuci elektricno napajanje
  - Postojana: memorije sa namagnetisanom povrsinom (informacije se ne gube nakon iskljucenja napajanja)
- Obrisiva/neobrisiva
  - Neobrisiva: memorija koja ne moze ni da se brise ni da se menja (npr. ROM (read-only memoty))

2. Metode pristupanja podacima memorije (sekvencijalno, direktno, nasumicno, asocijativno)

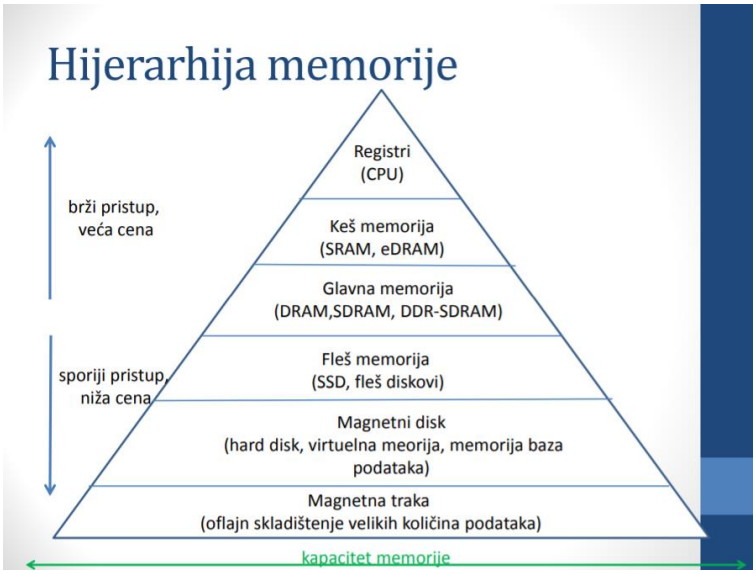
➔ Sekvencijalno pristupanje
- Pristupanje mora da se obavlja odredjenim linearnim redom
- Memorija je organizovana u jedinice podataka tzv. zapise
- Informacije o adresama se koriste za razdvajanje zapisa
- Zajednicki mehanizam za citanje i upisivanje, pri cemu se vrsi pomeranje sa trenutnog na zeljeni polozaj preskakanjem svih zapisa izmedju
- Vreme pristupa je promenljivo
- Magnetne trake koriste sekvencijalno pristupanje

➔ Direktno pristupanje
- Zajednicki mehanizam za citanje i upisivanje
- Pojedinacni blokovi ili zapisi imaju jedinstvenu adresu na osnovu fizicke lokacije
- Pristupanje se obavlja direktnim pristupom blizu trazenih podataka, a zatim se pretrazuje kako bi se stiglo do konacne lokacije
- Vreme pristupa je promenljivo
- Koriste ga magnetni diskovi

➔ Nasumicno pristupanje
- Svaka lokacija u memoriji koja moze da se adresira ima zaseban fizicki oziceni mehanizam za adresiranje
- Vreme pristupa za odredjenu lokaciju zavisi od prethodnih pristupa i nepromenljivo je
- Bilo koja lokacija moze da se direktno adresira i da joj se direktno pristupi
- Koriste ga glavna memorija i neki sistemi kes memorija

➔ Asocijativno pristupanje
- Tip memorije sa nasumicnim pristupom, poredjenjem zeljenih lokacija bita unutar reci pronalazi se podudarnost (ovo se radi za sve reci istovremeno)
- Rec se preuzima na osnovu dela njenog sadrzaja, a ne na osnovu njene adrese
- Vreme pristupa odredjenoj lokaciji ne zavisi od lokacije i prethodnih pristupa tj. nepromenljivo je
- Koriste ga kes memorije

3. Hijerarhija memorije – objasnjenje piramide memorijske hijerarhije



- ➔ Kretanjem naniže kroz hijerarhiju memorije dešava se sledede:
  - Smanjuje se cena po bitu
  - Povedava se kapacitet
  - Povedava se vreme pristupa
  - Procesor ređe pristupa memoriji
    - Ovo se postiže iskorišdenjem principa lokalnosti
- ➔ Manje, skuplje i brže memorije se dopunjavaju vedim, jeftinijim i sporijim memorijama
- ➔ Moguce je organizovati podatke duž ove hijerarhije memorije tako da procenat pristupa za svaki naredni nivo bude značajno manji od nivoa iznad
- ➔ Tipičan računarski sistem je opremljen hijerarhijom memorijskih podsistema, od kojih su neki unutrašnji (procesor im direktno pristupa), a neki su spoljašnji (procesor im pristupa preko U/I modula)

4. Princip lokalnsti: vremenska i prostorna lokalnost

Princip lokalnosti ili lokalnost pozivanja memorije – memorija koju procesor poziva tokom izvršavanja programa, kako za instrukcije tako i za podatke, teži tome da se grupiše

- primeri: programske petlje, potprogrami koji se ponavljaju; operacije sa nizovima, matricama

Dva oblika lokalnosti:

- ➔ Vremenska lokalnost – sklonost programa da u bliskoj bududnosti poziva one jedinice memorije koje je pozivao u neposrednoj prošlosti
- ➔ Prostorna lokalnost – sklonost programa da pristupa jedinicama memorije čije su adrese jedna blizu druge tj. odražava sklonost procesora da instrukcijama pristupa redom kao i da lokacijama podataka pristupa redom (npr. obrada tabele podataka)

Oba oblika lokalnosti odnose se i na instrukcije i podatke u memoriji: • Vremenska lokalnost podataka i vremenska lokalnost instrukcija • Prostorna lokalnost podataka i prostorna lokalnost instrukcija

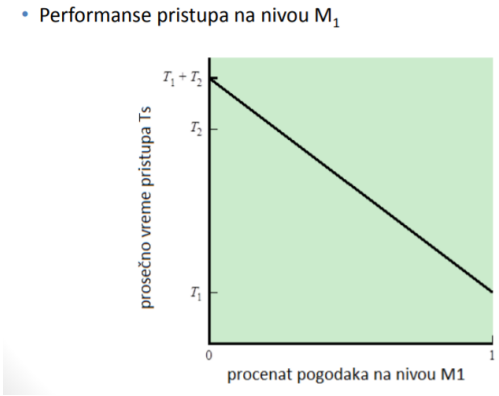
5. Clanovi hijrarhije memorije (registry, nivoi kes memorije, glavna memorija i sekundarna (virtuelna) memorija)
   - ➔ Registri u procesoru: najbrži, najmanji i najskuplji tip memorije
     - Tipično, procesor obuhvata nekoliko desetina registara, ali može i stotinak registara
     - Između CPU registara i glavne memorije stoji keš memorija
   - ➔ keš memorija je organizovana u više nivoa
     - Keš memorija nivoa 1, L1 keš memorija (Level 1 cache) – najbliža je registrima procesora
       - Skoro uvek podeljena na keš memoriju za instrukcije i keš memoriju za podatke
     - Keš memorija nivoa 2, L2 keš memorija
       - Sledeći nivo keš memorije
       - I za nju važi podela na keš memoriju za instrukcije i keš memoriju za podatke
     - L 3 keš memorija i L4 keš memorija
       - Obično se ne dele na zasebne delove za instrukcije i podatke
       - Mogu da budu zajedničke za više procesora
     - Keš memorije su tipično izrađene u SRAM (static random access memory) tehnologiji
     - Viši nivoi keš memorije se ostvaruju korišćenjem ugrađene dinamičke memorije sa nasumičnim pristupom eDRAM (embedded dynamic RAM)
   - ➔ Glavna memorija
     - Najvažnija unutrašnja memorija računara
     - Svaka lokacija ima jedinstvenu adresu
     - Vidljiva je programeru, dok keš memorija nije
     - Razne nivoe keš memorije kontroliše hardver i koristi se kao podrška premeštanju podataka između glavne memorije i registara procesora kako bi se poboljšale performanse
     - Registri, keš memorija i glavna memorija
       - Obično su to nepostojane memorije
       - Koriste poluprovodničku tehnologiju izrađenu u više tipova koje se razlikuju po brzini i ceni
     - Podaci se trajnije čuvaju na spoljašnjim uređajima za skladištenje velike količine podataka
       - Hard diskovi, magnetni diskovi i trake, optički diskovi
     - Spoljašnja, postojana memorija se naziva sekundarna (pomodna) memorija
   - ➔ Sekundarna memorija
     - Koristi se za čuvanje datoteka sa programima i podacima
     - Programerima je vidljiva u obliku datoteka i zapisa, ne u vidu pojedinačnih bajtova i reči
     - Hard diskovi se koriste kao proširenje glavne memorije poznato i kao virtuelna memorija
     - Pristupa joj se putem U/I modula
6. Pristupanje memorije sa dva i tri nivoa – princip i prosecno vreme pristupa

➔ Pristupanje memoriji sa dva nivoa

- Uzedemo kao najjednostavniji primer, koji se retko primenjuje u savremnim sistemima, računar koji ima samo jedan nivo keš memorije povezan sa glavnom memorijom
  - Keš memorija – kao posredna (eng.buffer) memorija između procesora i glavne memorije -> nastaju dva nivoa unutrašnje memorije
  - Mehanizam keširanja glavne memorije ostvaruje se u hardveru i obično je nevidljiv za operativni sistem
- Nivo memorije M1 je manji, brži i skuplji u odnosu na nivo memorije M2
- Nivo M1 se koristi kao privremeno skladište za deo sadržaja vedeg nivoa M2
- Kada se pozove memorija, prvo se pokuša pristupanje određenom podatku na nivou M1. Ako uspe, to je brz pristup
- Ako ne uspe, tada se blok memorijskih lokacija kopira sa nivoa M2 na nivo M1 , a zatim se traženom podatku pristupa na nivou M1
- Zbog principa lokalnosti, pošto se blok podataka preneo na nivo M1 , u narednom periodu očekuje se više pristupa lokacijama u tom bloku -> brži pristup
- Prosečno vreme pristupa nekom podatku:

$$Ts = H * T1 + 1 - H * (T1 + T2)$$

  - Ts- prosečno vreme pristupa za sistem
  - T1 - vreme pristupa za nivo M1 (npr. keš memorija)
  - T2 - vreme pristupa za nivo M2 (npr. glavna memorija)
  - H – stopa pogodaka (eng. hit). Pogodak je slučaj kada se podatak nađe u bržoj memoriji M1



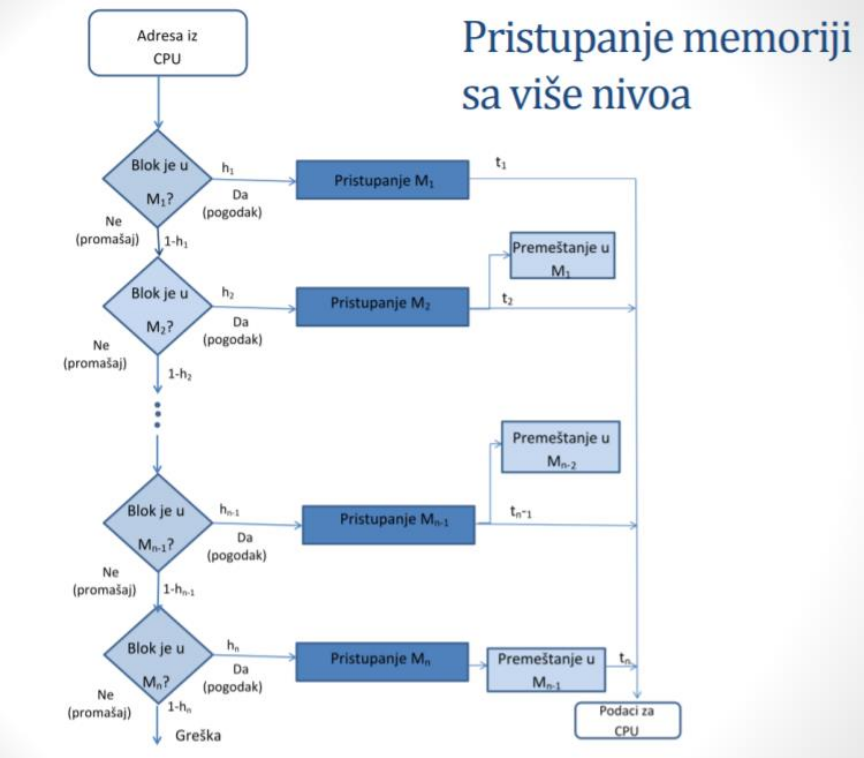
- Ako postoji jaka lokalnost mogude je ostvariti velike stope pogodaka i sa relativno malom veličinom keš memorije
- Keš memorija od 1K do 128K reči je uopšteno odgovarajuda, dok se glavna memorija izražava u gigabajtima
- Zaključak: relativno mali nivo M1 daje velike stope pogodaka zahvaljujudi lokalnosti

➔ Pristupanje memoriji sa vise nivoa

- Primer memorije sa 3 nivoa: jedan nivo keš memorije M1 , glavna memorija M2 i sekundarna memorija M3
- Prosečno vreme pristupa nekom podatku:

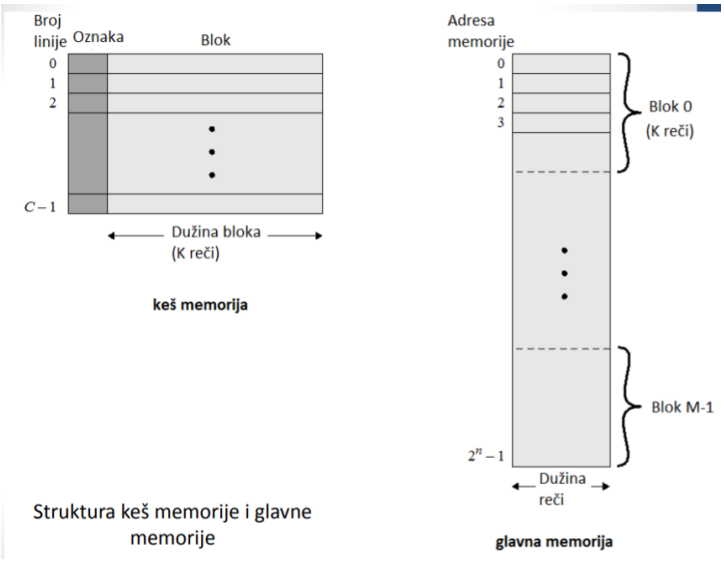
$$Ts = h1 * t1 + 1 - h1 * h2 * t2 + 1 - h1 * 1 - h2 * h3 * t3$$

- hi - stopa pogodaka na nivou Mi (uslovna verovatnoda da se podaci nalaze na nivou Mi pod uslovom da se ne nalaze na nivou Mi-1)
- ti - ukupno vreme potrebno za pristupanje podacima na nivou Mi . Jednako je zbiru svih vremena na putanji do pogotka na nivou Mi



II KES MEMORIJA

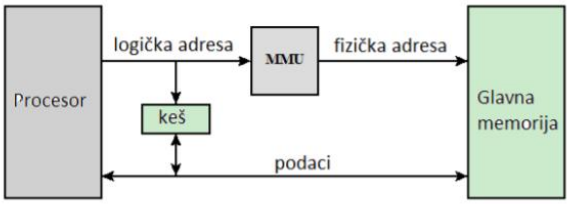
1. Struktura keš memorije i glavne memorije – pojam bloka, linije i oznake
   - Keš memorija obuhvata kopiju delova glavne memorije
   - Korišdenje više nivoa keš memorije – L2 keš memorija je sporija i obično veda od L1, a L3 je obično sporija i veda od L2 keš memorije
   - Blok: najmanja jedinica prenosa između keš memorije i glavne memorije
   - Linija: deo keš memorije koji može da drži jedan blok
   - Oznaka: deo linije keš memorije koji se koristi za adresiranje; linija keš memorije može da sadrži i druge kontrolne bitove



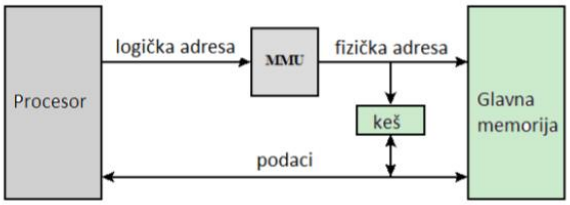
Struktura keš memorije i glavne memorije

- Glavna memorija sastoji se od najviše 2n reči koje mogu da se adresiraju, pri čemu svaka rečima jedinstvenu adresu od n bitova
- Zbog preslikavanja, ova memorija se posmatra kao skup više blokova nepromenljive dužine, od kojih svaki sadrži K reči
- Keš memorija sastoji se iz m linija, svaka linija sadrži K reči uz pridruženu oznaku
  - Linija sadrži i kontrolne bite npr. bit koji pokazuje da li je određena linija menjana otkako je učitana u keš memoriju
  - Tipična veličina linije je 128 bajtova
- Broj linija keš memorije značajno je manji od broja blokova
- glavne memorije tj. m<<M
- Kad se neka reč u bloku memorije pročita, tad se čitav blok prenosi u neku liniju keš memorije
- Svaka linija obuhvata oznaku kojom se navodi tačno određeni blok memorije koji se u njoj trenutno čuva -> ta oznaka je obično deo adrese glavne memorije

2. Čitanje iz keš memorije – pogodak i promašaj u keš memoriji
   - Procesor generiše adresu reči koja treba da se pročita
   - Ako se ta reč nalazi u keš memoriji ->pogodak u keš memoriji -> reč se isporučuje do procesora
   - Ako se desi promašaj u keš memoriji, dve stvari moraju da se dese:
     - Pristupanje glavnoj memoriji za blok koji sadrži traženu reč (adresu)
     - Dodeljivanje linije keš memorije za blok glavne memorije koji de se kopirati
     - 1. Princip „prvo najvažnija reč“ : tražena reč se isporučuje procesoru
     - 2. zatim se u keš memoriju učitava blok glavne memorije koji sadrži tu reč
   - Keš memorija je povezana sa procesorom preko linija za podatke, adrese i kontrolu
3. Logička i fizička keš memorija



(a) Logička keš memorija



(b) Fizička keš memorija

- Logička keš memorija:
  - Poznata i kao virtuelna keš memorija
  - Podatke skladišti upotrebom virtuelnih adresa
  - Procesor pristupa logičkoj keš memoriji direktno, bez prolaska kroz MMU
- Fizička keš memorija:
  - Podatke skladišti upotrebom fizičkih adresa glavne memorije
- Prednosti logičke keš memorija:
  - brži pristup podacima jer nema prevođenja adresa •
- Nedostatak logičke keš memorije: svim aplikacijama se uglavnom dodeljuje isti prostor virtuelnih adresa npr. virtuelna adresa 0
  - Potrebni dodatni bitovi da se označi na koji virtuelni prostor ukazuje adresa
  - Ili se keš memorija mora prazniti (eng. flushing cache) sa svakom promenom konteksta aplikacija

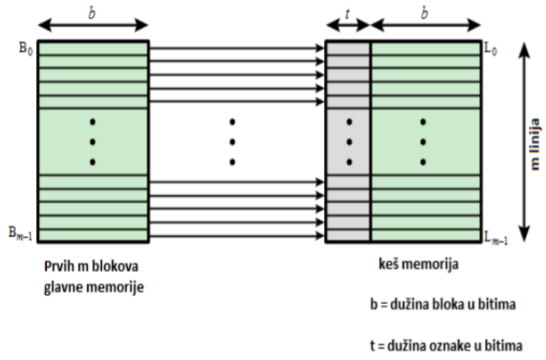
4. Tehnike preslikavanja iz glavne u keš memoriju:

➔ Direktno

- Svaki blok glavne memorije preslikava u tačno jednu mogudu liniju keš memorije koja mu je pridružena
- To preslikavanje se izražava sa:

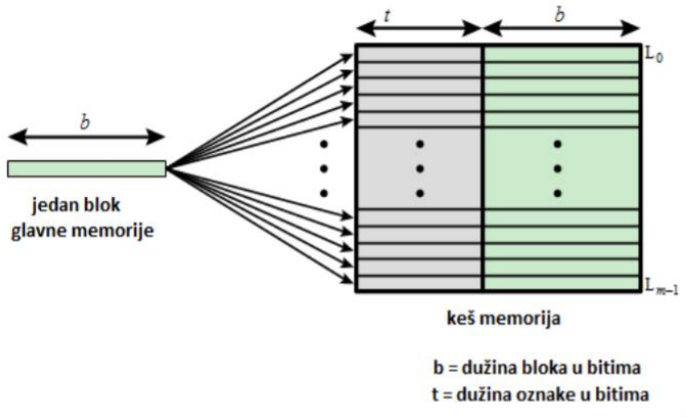
  $i = j \bmod m$

  - i je broj linije keš memorije
  - j je broj bloka glavne memorije
  - m je broj linija u keš memoriji
- Svaki blok glavne memorije preslikava se u jednu zasebnu liniju keš memorije. Sledeđih m blokova glavne memorije preslikava se u keš memoriju na isti način tj. blok Bm u liniju L0, blok Bm+1 u liniju L1 itd.



- Ova tehnika je jednostavna i nije skupa za primenu
- Mana: može se desiti da program više puta pristupa rečima iz dva različita bloka koji se preslikavaju u istu liniju, tako da ti blokovi stalno menjaju mesto u keš memoriji i stopa pogodaka je niska -> poznato kao treperenje (eng. trashing)

➔ Asocijativno
- Omogudava da se neki blok glavne memorije učita u bilo koju liniju keš memorije
- Da bi se utvrdilo da li se neki blok nalazi u keš memoriji , moraju se istovremeno pretražiti sve linije keš memorije da bi se našlo podudaranje
  - Za tu namenu koristi se CAM memorija (content-addressable memory) tj. memorija koja se adresira po sadržaju
- Postoji sloboda izbora koji blok se zamenjuje kad se novi blok učita u keš memoriju
- Nedostatak: složena logička kola neophodna za paralelno proveravanje svih oznaka linija keš memorije



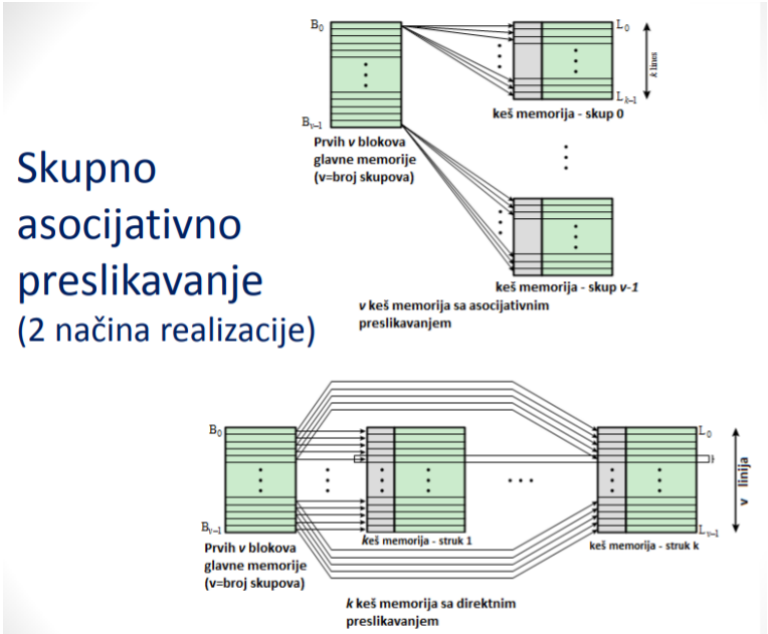
➔ Skupno asocijativno preslikavanje
- Koristi prednosti tehnike direktnog i asocijativnog preslikavanja dok umanjuje njihove nedostatke
- Keš memorija se posmatra kao da je organizovana iz više skupova pri čemu se svaki skup sastoji od više linija
- k-struko skupno asocijativno preslikavanje je definisano sa:

m = v*k

i = j mod v

- i je broj skupa keš memorije
- j je broj bloka glavne memorije
- m je broj linija u keš memoriji
- v je broj skupova
- k je broj linija u svakom skupu

- Može se realizovati na dva načina:
- 1. pomodu v keš memorija sa asocijativnim preslikavanjem (v CAM memorija)
  - Blok glavne memorije B0 se preslikava u skup 0, pri čemu se neka reč iz ovog bloka može preslikati u bilo koju liniju u skupu 0 • blok Bj može da se preslika u bilo koju od linija iz skupa j
- 2. pomodu k keš memorija sa direktnim preslikavanjem
  - Za svaku keš memoriju sa direktnim preslikavanjem kaže se da je struk koji sačinjava v linija
  - Prvih v linija glavne memorije direktno se preslikava u v linija nekog struka



5. Algoritmi zamene
   - Nakon što se keš memorija napuni, kad se novi blok dovede u keš memoriju jedan od postojedih blokova mora da se zameni
   - Kod direktnog preslikavanja postoji samo jedna linija za svaki blok i tu nema izbora pri zameni
   - Kod asocijativnog i skupnog asocijativnog preslikavanja neophodan je algoritam za zamenu blokova podataka
   - 4 najkorišdenija algoritma zamene:
   - Zamena najdavnije korišdenog bloka (eng. least recently usedLRU)

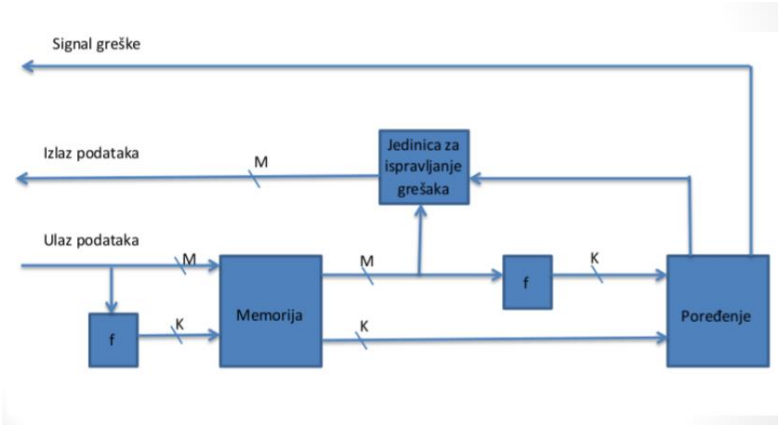
        - Zamenjuje se onaj blok u skupu koji se nalazi najduže u keš memoriji bez ukazivanja na njega
        - Svaka linija ima dodatni bit koji označava da li je korišdena
    - Zamena bloka u skupu koji je najduže u keš memoriji po FIFO principu (first in first out)
        - Raspoređivanjem blokova u krug
    - Zamena najređe korišdenog (eng. least frequently used-LFU)
        - svakoj liniji se pridružuje brojač pa se izbacuje onaj blok koji je najmanje puta upotrebljen
    - Nasumična zamena
6. Pravila upisivanja (istovremeno i odloženo upisivanje)
    - Ako stari blok u keš memoriji nije menjan on može da se prebriše novim blokom pri čemu stari blok ne mora pre toga da se prepiše u glavnu memoriju
    - Ako je na nekoj reči u liniji keš memorije obavljena bar jedna operacija upisivanja, tada mora da se ažurira glavna memorija upisivanjem te linije u blok glavne memorije (a pre nego što se donese novi blok) • postoji više pravila upisivanja
    - 1. istovremeno upisivanje – najjednostavnija tehnika
        - Sve operacije upisivanja se uporedo obavljaju u glavnoj memorji i keš memoriji -> sadržaj glavne memorije je uvek ažuran
        - Mana: značajan saobradaj između memorija
    - 2. odloženo upisivanje – smanjuje se broj upisivanja u memoriju
        - Ažuriranja se obavljaju samo u keš memoriji postavljanjem bita promene (ili bita korišdenja) koji je pridružen liniji
        - Kad se blok menja, on se upisuje u glavnu memoriju ako i samo ako je postavljen bit promene
        - Mana: delovi glavne memorije mogu biti nevaždedi sve dok se blok ne zameni

III GLAVNA MEMORIJA

1. Osobine i poređenje DRAM i SRAM memorije
    ➔ Osobine
    - Dinamička RAM memorija (DRAM):
        - Sačinjena je od delija koje čuvaju podatke u vidu naelektrisanja kondenzatora
        - Priroda kondenzatora je takva da se on polako prazni, pa DRAM memorije zahtevaju povremeno punjenje da bi se osvežilo stanje podatka
    - Statička RAM memorija (SRAM):
        - Za memorisanje podataka koriste se flip-flop logička kola
        - Tipična memorijska delija ima četiri tranzistora
        - Neophodno je napajanje električnom energijom, ali nema potrebe za osvežavanjem podataka
    ➔ Poredjenje
    - Obe su nepostojane tj. neophodno je električno napajanje

- Ćelija DRAM memorije je jednostavnija i manja -> DRAM memorija je gušda i jeftinija od SRAM
- Ali za DRAM memoriju su neophodna dodatna električna kola za osvežavanje
- SRAM memorije su nešto brže od DRAM memorija
- SRAM memorija se koristi za keš memoriju (i na čipu procesora i na zasebnom čipu)
- DRAM memorija se koristi za glavnu memoriju

2. Princip kodova za ispravljanje grešaka
   - Pri upisu podataka u memoriju obavlja se proračun na podacima -> predstavljen kao funkcija f kojom se dobija neki kod
   - Podaci i dobijeni kod se skladište u memoriji
     - M-bitna reč podatka + K-bitni kod = čuva se M+K bitova
   - Kad se čita prethodno sačuvana reč, ovaj kod se koristi za otkrivanje i mogude ispravljanje grešaka
     - Novi skup K-bitova koda se generiše iz M bitova podataka i poredi sa bitovima sačuvanog koda:
     - 1. greška nije pronađena -> preuzimaju se podaci i šalju
     - 2. greška je otkrivena i mogude je ispraviti grešku -> jedinica za ispravljenje grešaka
     - 3. greška je otkrivena i nije je mogude ispraviti -> stanje se prijavljuje



3. Hamingov kod – Hamingov kod za ispravljanje jedne greške (SEC kod) i Hamingov kod za ispravljanje jedne greške i otkrivanje dve greške (SEC-DED kod)

➔ Hamingov kod
   - Hamingov kod spada u najjednostavnije kodove za ispravljanje grešaka
   - Osmislio ga je Richard Hamming (Bell Laboratories)
   - Pri poređenju primenjuje se logičko ekskluzivno ILI nad dve Kbitne vrednosti koda
   - Rezultat toga je tzv. reč poremedaja
     - U njoj su 0 na mestima gde se ulazne vrednosti poklapaju, a 1 na mestima gde su različite
   - Pošto greška može da se desi na bilo kom od M+K bitova:

$$2K - 1 \geq M + K$$

   - Ova nejednačina govori koliko bitova provere (K) je potrebno za ispravljanje greške na jednom bitu u reči koja sadrži M bitova podataka

➔ Hamingov kod za ispravljanje jedne greške (eng. single error correcting – SEC)

- Za 8 bitova podataka potrebna su 4 bita provere $(2^4 - 1 \geq 8 + 4)$
- Bitovi provere se izračunavaju na slededi način:
  - $C_1 = D_1 \oplus D_2 \oplus D_4 \oplus D_5 \oplus D_7$
  - $C_2 = D_1 \oplus D_3 \oplus D_4 \oplus D_6 \oplus D_7$
  - $C_4 = D_2 \oplus D_3 \oplus D_4 \oplus D_8$
  - $C_8 = D_5 \oplus D_6 \oplus D_7 \oplus D_8$

Položaj bita	12	11	10	9	8	7	6	5	4	3	2	1
Broj položaja	1100	1011	1010	1001	1000	0111	0110	0101	0100	0011	0010	0001
Bit podatka	D8	D7	D6	D5		D4	D3	D2		D1		
Bit provere					C8				C4		C2	C1

- Primer: neka je ulazna reč sa podacima 00111001, pri čemu je D1 na krajnje desnom položaju
- Tada su biti provere: C1 =1, C2 =1, C4 =1, C8 =0
- Pretpostavimo da se bit podatka D3 oštetio i da je promenjen sa 0 na 1 (00111101)
- Onda su biti nove provere: C'1 =1, C'2 =0, C'4 =0, C'8 =0
- Reč poremećaja dobija se poređenjem bitova stare provere sa bitovima nove provere

	C8	C4	C2	C1
	0	1	1	1
⊕	0	0	0	1
→	0	1	1	0

Reč poremećaja: u ovom slučaju ukazuje da je greška na položaju 6 (0110). Taj položaj sadrži bit podatka D3.

➔ Poluprovodnička memorija je mnogo češde opremljena Hamingovim kodom za ispravljanje jedne greške i otkrivanje dve greške (eng. single error correcting, double error detecting, SEC-DED)

- Radi na istim principima, ali u odnosu na Hamingov SEC kod potreban je jedan bit više u bitovima provere
- Kod za ispravljanje grešaka povedava pouzdanost memorije na račun povedane složenosti: savremeni sistemi DRAM memorije imaju 7-20% dodatne memorije

	SEC kod (ispravljanje 1 greške)	SEC-DED kod (ispravljanje 1, detekcija 2 greške)
Bitovi podataka	Biti provere	Biti provere
8	4	5
16	5	6
32	6	7
64	7	8
128	8	9
256	9	10

4. Sinhrona DRAM (SDRAM) memorija
   - Postoji više poboljšanja osnovne arhitekture DRAM memorije, a najkorišdenije su SDRAM i DDR-DRAM memorija

- Za razliku od tradicionalne, asinhrone DRAM memorije, sinhrona DRAM (SDRAM) memorija razmenjuje podatke sa procesorom sinhrono sa signalom spoljašnjeg sistemskog sata
- Radi punom brzinom procesora/ memorijske sabirnice bez stanja čekanja
- Sinhronim pristupom, sistemski sat kontroliše premeštanje podataka iz/u DRAM memoriju
    - Procesor izda instrukcije i adresu koje preuzima SDRAM memorija
    - Memorija odgovora posle određenog broja ciklusa sistemskog sata
    - U međuvremenu, procesor obavlja druge zadatke, dok SDRAM obrađuje zahtev
- SDRAM memorija koristi režim protočne obrade
    - Niz bitova podataka može brzo da se obradi sinhrono sa sistemskim satom pošto se pristupi prvom bitu
    - SDRAM memorija obuhvata više memorijskih grupa koje povedavaju mogudnost za paralelnu obradu
    - Korisno pri prenosu velikih blokova podataka npr. programi za obradu teksta, tabelarna izračunavanja i multimedijalne aplikacije
- Registar režima rada – novina kod SDRAM memorije
    - Određuje dužinu protočne obrade tj. broj zasebnih jedinica podataka koje se sinhrono prenose

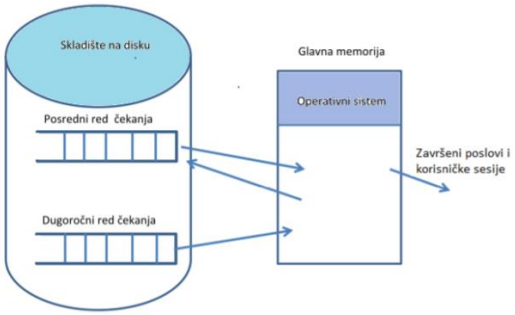
5. DDR SDRAM memorija
    - To je SDRAM memorija dvostruke brzine prenosa podataka (eng. double data rate, DDR)
    - DDR memorija postiže vede brzine prenosa na tri načina:
        - 1. prenos se sinhronizuje i sa uzlaznom i sa silaznom ivicom signala sistemskog sata -> udvostručena brzina prenosa
        - 2. koriste se veda brzina radnog takta na sabirnici
        - 3. koristi se privremeno skladištenje podataka u tzv. prihvatnoj memoriji za prethodno preuzimanje
    - Do sada su izašle 4 generacije tehnologije DDR memorije
        - DDR1 – 2-bitna prihvatna memorija,
        - DDR2 – 4-bitna prihvatna memorija
        - DDR3 – 8-bitna prihvatna memorija
        - DDR4 – 8-bitna prihvatna memorija + memorijske grupe
    - Prihvatna memorija omogudava da se paralelno pristupa podacima, a zatim se oni kroz multiplekser prenose serijski izvan memorije
    - Brzine prenosa u odnosu na klasični SDRAM:
        - DDR1 :2x brže
        - DDR2: 4x brže
        - DDR3: 8x brže
        - DDR4: oko 12x brže
            - ciklus kolone se završava unutar memorijske grupe, kojih može biti do 4
            - Npr. dva preuzimanja od po 8 bita se obavljaju paraleleno u dve memorijske grupe

• Zadržana ista veličina prihvatne memorije kao kod DDR3, ali povedane performanse

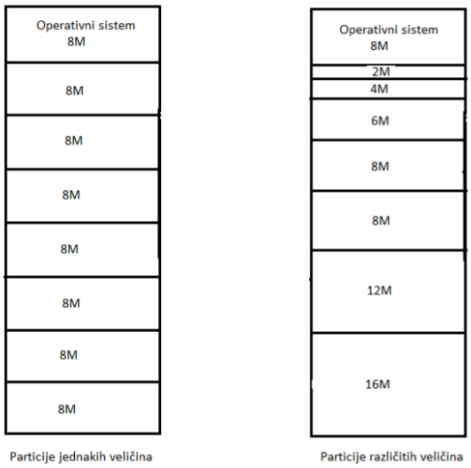
IV UPRAVLJANJE MEMORIJOM

1. Operativni sistem za višeprogramski rad – upravljanje memorijom i raspoređivanje procesa
   - Kod jednostavnog paketnog OS, procesor je često bez posla zbog činjenice da su U/I uređaji mnogo sporiji u poređenju sa procesorom
   - pretpostavimo da u memoriji ima dovoljno mesta za OS i dva korisnička programa
   - Dok jedan program čeka na U/I operaciju, procesor može da preskoči na drugi posao koji ne čeka na U/I operaciju
   - Možemo dalje da proširimo memoriju tako da sadrži tri, četiri li više programa i da se menja između njih
   - Ova tehnika se zove višeprogramski rad (multiprogramming)
   - Kod višeprogramskog rada hardver ima osobinu da podržava U/I prekide i direktno pristupanje memorije (DMA)
   - OS za višeprogramski rad su znatno složeniji u poređenju sa OS za jednoprogrmaski rad
   - Za OS za višeprogramski rad neophodno je:
     - Upravljanje memorijom – pošto se više programa moraju držati u glavnoj memoriji
     - Planiranje i raspoređivanje procesa (programa) – procesor odlučuje koji od spremnih poslova će da pokrene
2. Upravljanje memorijom – svrha i tehnike upravljanja
   - U sistemu za višeprogramski rad, u glavnoj memoriji je uz OS, smešteno više procesa u „korisničkom“ delu memorije
   - Zadatak deljenja memorije u manje celine OS obavlja dinamički i taj proces se zove upravljanje memorijom
   - Efikasno upravljanje memorijom je izuzetno važno u sistemu za višeprogramski rad
   - Tehnike upravljanja memorijom:
     1. Zamena mesta
     2. Particionisanje memorije
     3. Straničenje
     4. Segmentacija memorije
3. Zamena mesta (swapping)
   - Dugoročni red čekanja zahteva za proces se obično skladišti na disku
   - Zahtevi za proces se prebacuju u memoriju kada za to ima mesta u njoj
   - Ako nastane situacija da nijedan od procesa u memoriji nije spreman za obradu (npr. svi čekaju U/I operacije), onda procesor vrši zamenu mesta nekom od tih procesa vraćajući ga na disk u posredni red čekanja
   - Posredni red čekanja – je red čekanja postojećih procesa koji su privremeno izbačeni iz memorije
   - OS dovodi drugi proces ili iz posrednog reda čekanja ili ispunjava zahtev za novi proces iz dugoročnog reda čekanja

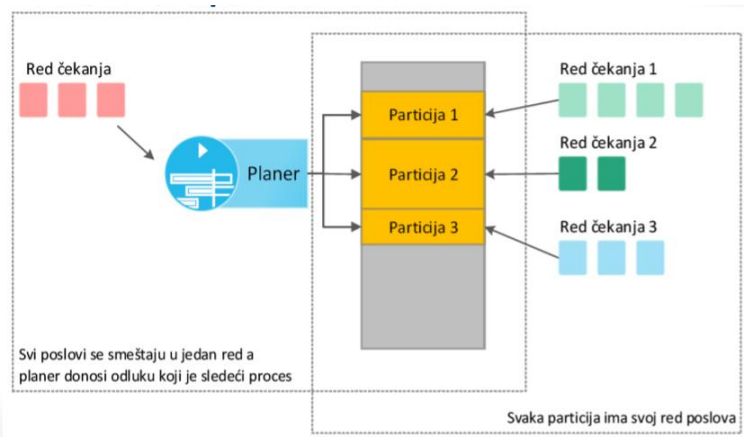
- Zamena mesta je U/I operacija pa postoji mogućnost da se problem pogorša
- Koristi se u prioritetnim šemama za raspoređivanje procesa
    - Procesi niskog prioriteta se smeštaju na disk i čekaju da se oslobodi memorija



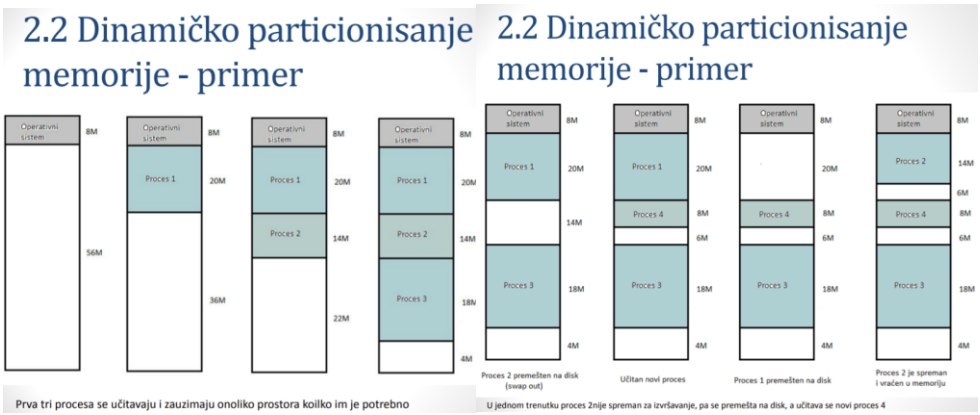
4. Statičko particionisanje memorije - objašnjenje i primer
    - Osnovna ideja je da se memorija podeli na n particija koje predstavljaju neprekidne delove memorije.
    - Svaka particija je nepromenljive veličine, ali ne moraju sve particije biti jednake veličine
    - Dakle, veličine particija su fiksne i najčešće nisu jednake veličine
    - U svaku particiju se može smestiti po jedan proces
    - Ako su sve particije zauzete, a neki program treba da započne izvršavanje, onda se formira red spremnih procesa



- Kada se neka od zauzetih particija oslobodi, učitava se neki od spremnih procesa
- Postoje dve strategije za odabir particije u koju bi proces trebalo da bude smešten:
    - 1. Svaka particija ima svoj red čekanja u koji se smeštaju procesi čiji memorijski zahtevi odgovaraju veličini te particije
    - 2. Svi poslovi se smeštaju u jedan red čekanja, a dispečer (planer) donosi odluku koji je sledeći proces koji bi trebalo da se izvrši.
- nedostatak: u većini slučaja procesu ne treba baš onoliko memorije koliko je velika particija (npr. proces od 3MB u particiji od 4MB) -> neiskorišćenost memorije

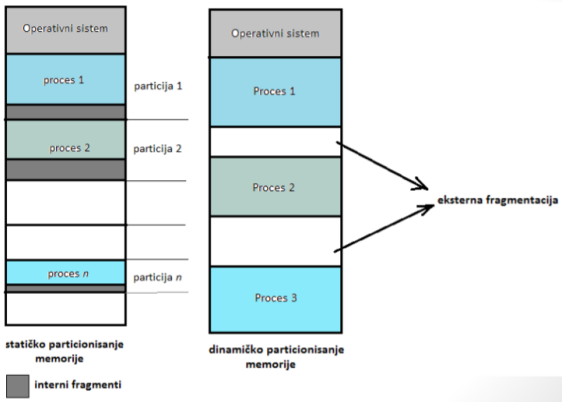


5. Dinamičko particionisanje memorije – objašnjenje i primeri; načini vođenja evidencije
   - U ovom slučaju veličine particija nisu fiksne, već kada se neki proces unosi u memoriju dodeljuje mu se tačno onoliko memorije koliko mu treba
   - Sistemi koji podržavaju dinamičke particije moraju voditi računa o slobodnoj i zauzetoj memoriji
   - Kada novi proces dođe na red za izvršavanje, pronalazi se dovoljno velika slobodna particija, a zatim se iz nje izdvaja onoliko memorije koliko je procesu potrebno
   - Nakon završetka izvršavanja, proces oslobađa particiju koju je koristio



   - Šupljina je termin koji označava slobodni, kontinualni deo memorije
   - Kada naiđe proces, sistem traži šuljinu dovoljno veliku da smesti čitav proces
   - OS vodi evidenciju o:
     - Particijama koje su zauzete procesima
     - Slobodnim šupljinama u memoriji
   - Načini vođenja evidencije:
     - Bit mape
     - Povezane liste
   - Bit mape podrazumevaju da se memorija podeli na jednake delove i da se svakom od tih delova pridruži po jedan bit
     - Pridruženi bit ima vrednost 1 ukoliko je odgovarajući deo memorije zauzet, u suprotnom je 0

    - Za smeštanje procesa u memoriju potrebno je pronaći dovoljno veliki niz nula u bit mapi
    - Prednost: implementacija bit mapa je jednostavna
    - Mana: vreme potrebno za pretraživanje i pronalaženje slobodne particije dosta veliko
  - Evidencija u povezanoj listi se sastoji od čvorova, pri čemu svaki čvor sadrži sledeće:
      - Oznaka tipa memorije - da li je u delu memorije, na koji se čvor odnosi, smešten proces (P) ili je u pitanju slobodna memorija (H)
      - Početna adresa odgovarajućeg dela memorije
      - Dužina tog dela memorije
      - Pokazivač na sledeći čvor koji čuva informacije o narednom delu memorije
    - U nekim slučajevima se liste slobodnih i zauzetih delova memorije implementiraju odvojeno
      - Time je olakšana pretraga slobodnih delova, ali je mana ažuriranje dve liste
6. Algoritmi za dodelu memorije kod particionisanja
  - Postoji više strategija pri izboru slobodne šupljine u koju se može smestiti proces
  - 1. prvo podudaranje (first fit)
    - Pretraga ide od početka liste ili u podvarijanti od mesta gde je stala prethodna pretraga (next fit)
    - Pretraga se zaustavlja kad se naiđe na prvu šupljinu koja odgovara
    - Vreme pretraživanja je minimalno, nema pretrage cele liste
    - Česta pojava fragmentacije
  - 2. najbolje podudaranje (best fit)
    - Procesu se dodeljuje najmanja šupljina (od svih dostupnih) u koju može stati proces
    - Pretražuje se cela lista -> intenzivna pretraga, vremenski zahtevnije, ali ukupna iskorišćenost memorije bolja
7. Fragmentacija memorije – tipovi fragmenatcije i način prevazilaženja tog problema (tehnika sabijanja memorije)
  - Fragmenti su slobodni memorijski prostori koje sistem ne može iskoristiti, iako postoje procesi koji čekaju da se izvrše
  - Fragmentacija se javlja u dva oblika:
    - 1. Interna - javlja se kod statičkog particionisanja memorije i odnosi se na preostali neiskorišćen prostor koji ostaje kada se proces smesti u veću particiju
    - 2. Eksterna - u toku procesa dinamičke particije kada su sve slobodne lokacije zajedno dovoljne za izvršavanje procesa, ali ne formiraju jedinstvenu, kontinualnu particiju



8. Adrese u memoriji – fizičke i logičke adrese; jedinica za upravljanje memorijom (MMU); registar relokacije
   - Prilikom zamene mesta neki proces se verovatno neće učitati na isto mesto kad se vrati nazad u memoriju (kao Proces 2 u primeru za dinamičko particionisanje)
   - Dodatno, ako se koristi sabijanje neki proces se može pomeriti dok je u glavnoj memoriji
   - Proces u memoriji se sastoji od instrukcija i podataka
   - kod sistema sa višeprogramskim radom, veći broj procesa deli operativnu memoriju računara
     - programer ne može unapred odrediti fiksne memorijske lokacije za smeštaj programa
     - adrese podataka i instrukcija su promenljive jer se menjaju svaki put kad se proces premešta
       -> rešenje je uvođenje razlike između logičkih adresa i fizičkih adresa

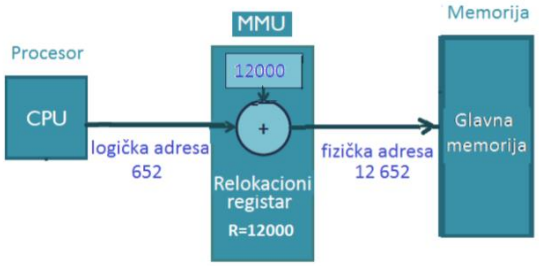
➔ Logička adresa: izražava se kao relativni položaj u odnosu na početak programa
   - Instrukcije u programu sadrže samo logičke adrese
   - Logičke adrese su generisane od strane procesora dok se program izvršava
     - Naziva se još i virtualna adresa jer ona fizički ne postoji
   - logički (virtualni) adresni prostor je skup svih logičkih adresa koje generiše jedan program

➔ Fizička adresa: stvarna adresa u memoriji
   - Identifikuje fizičku lokaciju u memoriji
   - Procesor pri izvršavanju procesa automatski pretvara logičke adrese u fizičke adrese tako što na svaku logičku adresu dodaje trenutni početni položaj procesa koji se naziva osnovna adresa (base address)

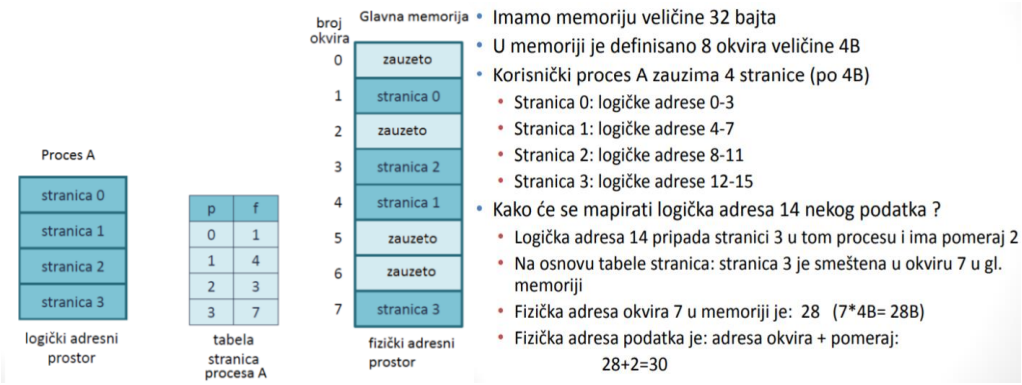
➔ Jedinica za upravljanje memorijom (Memory Management Unit, MMU) preslikava logičke adrese u fizičke adrese prilikom izvršavanja programa
     - MMU je hardverski uređaj koji mapira virtuelni adresni prostor u fizički adresni prostor
   - Jednostavni MMU se može predstaviti kao uređaj koji poseduje poseban registar koji se naziva registar relokacije
     - Korisnički program barata logičkim adresama, koje on vidi u opsegu od [0, max]

- MMU dodaje vrednost registra relokacije (R) na logičku adresu i tako generiše fizičku adresu
  • Vrednost registra relokacije R je trenutna fizička adresa početka programa u memoriji
- MMU preslikava logički adresni prostor koji se nalazi u opsegu [0, max] u fizički adresni prostor u opsegu [R+0, R+max]

- MMU i registar relokacije
- R označava vrednost registra relokacije tj. fizičku adresu početka programa u memoriji
- Fizička adresa = logička adresa + R



9. Straničenje memorije – osnovni principi; pojam okvira, stranice i tabele stranica; primer; prednosti
   - Memorija se deli na jednake parčiće nepromenljive veličine koji su relativno mali -> okviri ili okviri stranice (page frames)
   - svaki proces se deli na parčiće iste nepromenljive veličine -> to su stranice (pages)
   - Tada te parčiće programa tj. stranice smeštamo u raspoložive okvire
   - Ova metoda ima hardversku podršku na nivou procesora
   - OS drži spisak slobodnih okvira
   - Primer:
     - Neka proces A koji se čuva na disku sačinjavaju 4 stranice
     - Kada treba učitati program A u memoriju, OS pronalazi 4 slobodna okvira i u njih smešta 4 stranice procesa A



- Kao što se vidi iz prethodnog primera iako u memoriji nije bilo dovoljno slobodnih, uzastopnih okvira za smeštanje procesa A, to nije bila prepreka da OS učita proces u slobodne okvire koji su razbacani u memoriji

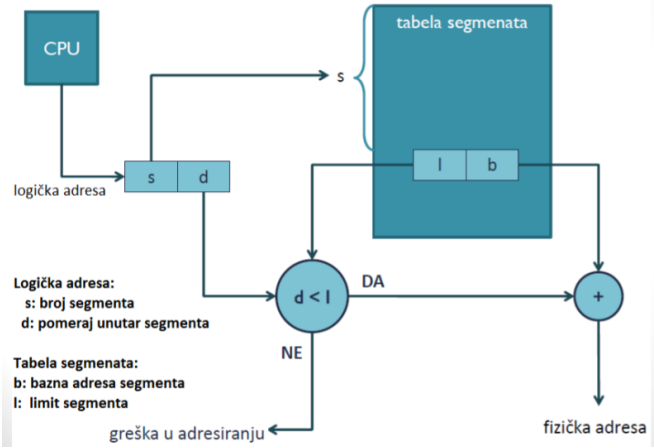
- Kao posledica ovog načina smeštanja procesa, OS mora da vodi tabelu stranica za svaki proces
- tabela stranica pokazuje položaje okvira za sve stranice procesa
  - Tabela stranica nam govori u koji okvir je smeštena neka stranica procesa
- Kada se neki proces unosi u memoriju, njegove stranice se učitavaju u raspoložive okvire i popunjava se tabela stranica

10. Straničenje memorije i adresiranje – objašnjenje sadržaja logičke adrese i njeno prevođenje u fizičku adresu
    - U slučaju straničenja memorije, logička adresa obuhvata dva podatka: broj stranice i relativnu adresu unutar te stranice
    - Prevođenje logičke u fizičku adresu i dalje obavlja procesor
    - Procesor mora da zna kako da pristupi tabeli stranica tekućeg procesa
    - Procesor korišćenjem logičke adrese (broj stranice, relativna adresa unutar stranice) :
    - 1. prvo pristupa tabeli stranica datog procesa da bi za dati broj stranice saznao broj okvira u koji je stranica smeštena
    - 2. zatim pristupa podatku u glavnoj memoriji koristeći dati broj okvira memorije i relativnu adresu unutar stranice (okvira)
11. Straničenje po zahtevu. Virtuelna memorija.
    - Korišćenjem straničenja, efikasnost sistema za višeprogramski rad dolazi do punog izražaja
    - Straničenje po zahtevu (demand paging) označava da se određena stranica učitava u memoriju samo kad je potrebna tj. na zahtev
    - Zamislimo slučaj velikog procesa koji se sastoji od dugačkog programa i više nizova podataka. Tokom nekog kraćeg vremenskog perioda izvršavanje može da bude ograničeno na manji deo programa (npr. neki potprogram) i možda jedan niz podataka
    - Učitavanje desetina stranica procesa bilo bi neefiksano trošenje resursa. Umesto toga možemo učitati tek nekoliko stranica procesa
    - Zatim ako se program grana na instrukciju na stranici koja nije u glavnoj memoriji ili ukazuje na podatke koji nisu u memoriji, pokreće se prekid nazvan greška stranice kojim se OS-u saopštava da učita traženu stranicu
    - Tako se u bilo kom trenutku u memoriji nalazi tek nekoliko stranica svakog od procesa i zato je moguće držati više procesa u memoriji
    - Štedi se i vreme jer se nekorišćene stranice ne premeštaju u memoriju niti iz memorije
    - Kada se neka nova stranica učita u memoriju, mora se neka stara stranica izbaciti -> zamena stranica
    - Algoritmi za zamenu starnica
      - LRU –algoritam najdavnije korišćene stranice (kao kod keš memorije)
      - U praksi se LRU teško realizuje za straničenje memorije, već se koriste malo drugačiji algoritmi
    - Straničenjem po zahtevu moguće je da proces bude veći od čitave glavne memorije

        - -> programer ne brine da li mu je program suviše velik tj. da li ima dovoljno memorije
        - Bez straničenja, programer bi morao da realizuje svoj veliki program iz delova koji bi se jedan po jedan učitavali u memoriju
        - -> straničenjem po zahtevu taj posao je sad prepušten OS
        - -> straničenjem po zahtevu, programer "vidi" ogromnu memoriju koja je jednaka prostoru koji joj je dodeljen na disku
    - Procesi se izvršavaju samo u glavnoj memoriji -> realna memorija
    - Programeri imaju utisak da je memorija mnogo veća, onoliko koliko je prostora za nju izdvojeno na disku -> virtuelna memorija
    - Virtuelna memorija omogućava efikasan višeprogramski rad i oslobađa korisnike ograničenja glavne memorije
12. Struktura tabele stranica. Koraci pri prevođenju logičke u fizičku adresu. Organizovanje velikih tabela stranica.
    - Zavisno od veličine procesa tabele stranica su promenljive dužine -> nije objektivno držati ih u registrima procesora
    - Umesto toga, tabele stranica su u glavnoj memoriji
    - Koraci koji procesor izvršava pri prevođenju logičke u fizičku adresu:
        - 1. Dok se neki proces izvršava, registar drži početnu adresu tabele stranica za taj proces
        - 2. Broj stranice iz logičke adrese se koristi kao indeks za tabelu stranica i pronalaženje broja okvira
        - 3. Tom pronađenom broju okvira pridodaje se deo logičke adrese koji označava pomeraj (relativnu adresu unutar stranice) i tako se konačno dobija fizička adresa
    - Šema sa dva nivoa za organizovanje velikih tabela stranica
        - 1. varijanta:
            - Postoji adresar stranica u kom svaki unos ukazuje na neku tabelu stranica
            - Za adresar stranica dužine X, pri čemu je najveća dužina tabele stranica Y -> proces može da se sastoji od X*Y stranica
        - 2. varijanta –korišćenje strukture invertovane tabele stranica
13. Struktura invertovane tabele stranica.
    - Deo virtuelne adrese koji predstavlja broj stranice preslikava se pomoću jednostavne heš funkcije u heš vrednost
        - Heš funkcija preslikava brojeve iz opsega [0,M] u opseg [0,N] pri čemu je M>N
        - Rezultat heš funkcije se koristi kao indeks heš tabele
        - Više ulaznih vrednosti može da se preslika u istu izlaznu vrednost -> tada se ta nova stavka smesti u susedni prazni prostor i doda se pokazivač iz prvobitne lokacije kojim se ti unosi uvezuju u lanac
    - Heš vrednost je pokazivač na invertovanu tabelu stranica koja sadrži unose za tabelu stranica

    - Za svaki okvir stranice u fizičkoj memoriji postoji jedan unos u invertovanoj tabeli stranica umesto jednog unosa po virtuelnoj stranici -> zato je tabela invertovana jer se unosi u tabeli stranica indeksiraju po broju okvira, a ne po broju virtuelne stranice
  - Za invertovanu tabelu stranica potreban je uvek isti deo stvarne memorije bez obzira na broj procesa ili na broj podržanih virtuelnih stranica
  - Pošto više virtuelnih adresa može da se preslika u isti unos u heš tabelu, koristi se tehnika povezivanja dodatnih unosa u lanac. Lanac je obično kratak (jedan do dva unosa)
14. Pomoćna memorija za preslikavanje adresa (TLB).
  - Svaka referenca na virtuelnu memoriju može da dovede do dva pristupanja fizičkoj memoriji:
    - 1. preuzimanje odgovarajućeg unosa iz tabele stranica
    - 2. preuzimanje željenih podataka
  - Iz ovoga sledi da bi jednostavna šema virtuelne memorije za posledicu imala udvostručavanje vremena pristupa memoriji
  - Iz tog razloga, većina šema virtuelne memorije koristi posebnu keš memoriju za unose iz tabele stranica
  - Ova keš memorija se najčešće naziva pomoćna memorija za preslikavanje adresa (translation lookaside buffer, TLB)
  - TLB keš memorija funkcioniše kao i obična keš meorija tj. sadrži one unose iz tabele straničenja koji su nedavno korišćeni -> po principu lokalnosti većina referenci virtuelne memorije biće na lokacije u nedavno korišćenim stranicama
15. Segmenatcija memorije – osnovni principi, logička adresa; tabela segmenata; mapiranje logičke u fizičku adresu
  - straničenje memorije je nevidljivo za programera i služi da omogući veći adresni prostor
  - Segmentacija je obično vidljiva za programere i nudi se kao pogodnost za organizovanje programa i podataka, kao i dodeljivanje privilegija i atributa zaštite instrukcijama i podacima
  - Segmentcija omogućava programerima da memoriju vide kao da se sastoji od više adresnih prostora ili segmenata • Segmenti su promenljive tj. dinamičke veličine
  - Svakom segmentu mogu da se dodele prava za pristupanje i korišćenje
  - Adrese za pristup memoriji su u obliku:
    - (Broj segmenta, pomeraj)
  - korisnički program se može posmatrati kao skup različitih logičkih celina – segmenata
    - Na primer, neki od njih su: glavni program, stek, hip, biblioteke, globalni podaci itd.
  - Osnovna ideja segmentacije je da se svakom segmentu dodeli poseban memorijski prostor
  - Segmenti se mogu naći bilo gde u memoriji, ali jedan segment mora biti u neprekidnom memorijskom bloku
  - Nije neophodno da segmenti budu iste veličine, već se svakom segmentu dodeljuje tačno onoliko memorije koliko je potrebno

- logički adresni prostor se sastoji od skupa segmenata
- svaki segment ima jedinstveno ime (ID) i dužinu
- logička adresa se se sastoji od dva dela:
  - Broj segmenta (broj koji predstavlja ID segmenta)
  - pomeraja unutar segmenta
- kod segmenatacije je prisutna eksterna fragmentacija
- slobodna memorija se ne može iskoristiti za smeštaj segmenta ukoliko ne postoji dovoljno velika šupljina
- problem se može umanjiti sažimanjem memorije
- kod metode segmentacije logičke adrese su dvodimezionalne
- mapiranje u fizičke adrese se obavlja preko tabele segmenata
- svaki ulaz u tabeli segmenata sadrži dva parametra:
  - baznu adresu segmenta (početna fizičku adresu segmenta u memoriji)
  - ograničenje (limit) segmenta (tj. dužina segmenta)
- mapiranje logičke adrese u fizičku se vrši na sledeći način:
  - broj segmenta (s) se koristi kao ulaz u tabelu segmenta
  - iz tabele se čitaju bazna adresa segmenta (b) i limit (l)
  - ako je pomeraj (d) veći od limita (d > l) -> greška u adresiranju
  - ako je pomeraj manji od limita (d < l): -> fizička adresa = bazna adresa + pomeraj

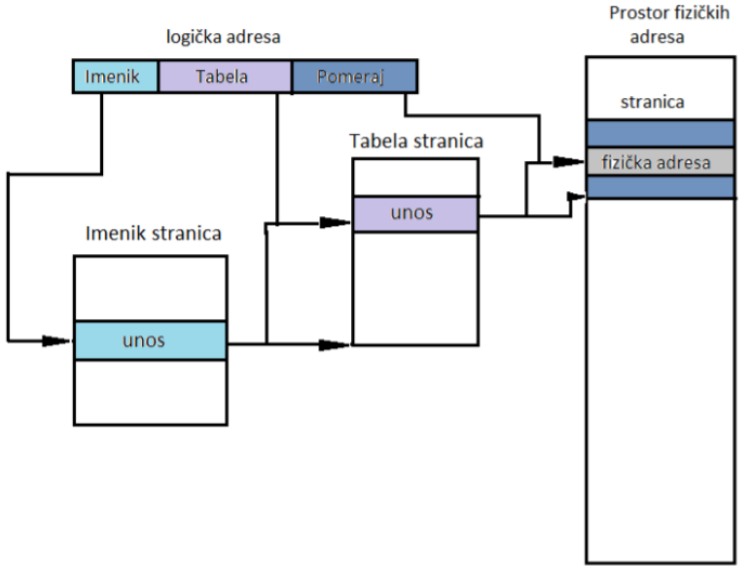


16. Segmentacija memorije – poređenje sa drugim tehnikama; prednosti
    - Dodeljivanje memorije je slično kao i kod straničenja
      - Razlika je u tome što su segmenti različite veličine
    - Potrebno je pronaći i dodeliti memoriju za sve segmente korisničkog programa
    - Dodela memorije se svodi na dinamičku dodelu memorije i rešava se prethodno opisanim algoritmima
    - Veličina segmenata je različita, pa segmentacija podseća na dinamičko particionisanje
      - Razlika između njih je u tome što se kod segmentacije proces ne nalazi u jednom memorijskom bloku, već se delovi procesa mogu naći bilo gde u memoriji
      - Kao i dinamičko particionisanje, segmentacija rešava problem interne fragmentacije ali ostaje problem eksterne fragmentacije

    - kod segmentacije, eksterna fragmentacija je manje izražena jer su segmenti manji od kompletnog programa, pa se lakše mogu „umetnuti“
  - Prednosti segmentacije memorije:
  - Pojednostavljuje se obrada rastućih struktura podataka
  - Moguće je da se programi nezavisno menjaju i ponovo kompajliraju, pri čemu nije neophodno da se čitava grupa programa ponovo učitava i uvezuje
  - Moguće je deljenje adresnog prostora između procesa npr. u nekom segmentu može biti pomoćni program kome mogu i drugi procesi da pristupaju
  - Olakšana je zaštita podataka – programer ili administrator lakše dodeljuju prava pristupa nekom segmentu u kome je tačno definisani skup programa ili podataka
17. Segmentacija memorije kod Intel x86 – virtuelne adrese, oblici zaštite segmenata (nivo prava i atribut pristupa)
  - Segmentacija memorije:
  - Svaka virtuelna adresa (u dokumenatciji x86 procesora ona se naziva logička adresa) sastoji se od:
    - 16-bitna referenca na segment
    - 32-bitni pomeraj
    - 2 bita iz reference na segment koriste se kao zaštitni mehanizam
  - Dakle, sa nesegmentiranom memorijom i 32-bitnim adresama u fizičkom adresnom prostoru, korisnička virtuelna memorija je $2^{32} = 4GB$
  - Sa segmentiranom memorijom, prostor za virtuelnu memoriju je $2^{14} * 2^{32} = 2^{46} = 64TB$
  - Svakom segmentu pridružena su dva oblika zaštite:
    - Nivo prava (privilege level)
    - Atribut pristupa (access atribute)
  - ukupno postoje 4 nivoa prava:
    - Najzaštićeniji nivo (nivo 0)
    - Najmanje zaštićen (nivo 3)
  - Nivo prava za segment podataka je njegova klasifikacija
    - Nivo prava za segment programa je njegovo odobrenje
    - Program koji se izvršava može da pristupa samo onim segmentima podataka koji imaju niži (program ima viša prava) ili jednak nivo prava
    - Nivo prava 0 koristi manji deo OS namenjen upravljanju memorijom, zaštiti i kontroli pristupa
    - Nivo prava 1 koristi najveći deo OS K
    - orisničke aplikacije najčešće koriste nivo 3
    - Nivo 2 – aplikacije posebne namene koje moraju da budu zaštićene npr. upravljanje bazama podataka, sistemi za automatizaciju
  - Mehanizam određivanja nivoa prava određuje pristup segmentima podataka, ali i uvodi ograničenja na korišćenje određenih instrukcija

- Neke instrukcije se izvršavaju samo na nivou 0: instrukcije za registre koji upravljaju memorijom
- U/I instrukcije se izvršavaju samo do određenog nivoa koji određuje OS, a to je obično nivo 1
- Atribut pristupa:
  - Atribut pristupa pridružen segmentu podataka određuje da li je dopušteno čitanje i upisivanje ili samo čitanje podataka
  - Atribut pristupa pridružen segmentu programa određuje pristupanje za čitanje i izvršavanje ili samo za čitanje

18. Straničenje memorije kod Intel x86 procesora
    - Mehanizam straničenja u x86 procesoru je operacija pretraživanja tabele u dva nivoa
    - Prvi nivo je imenik stranica koji sadrži do 1024 unosa
      - Ovim se prostor od 4GB deli na 1024 grupe stranica pri čemu svaka grupa ima svoju tabelu stranica dužine 4MB
    - Drugi nivo je tabela stranica
      - Svaka tabela stranica sadrži do 1024 unosa
      - svaki unos u tabeli odgovara jednoj stranici od 4 K bajta
    - Upravljanje memorijom, može da koristi:
      - jedan imenik stranica za sve procese, ili
      - jedan imenik za svaki proces, ili
      - njihovu kombinaciju



- Imenik stranica za tekući proces se uvek nalazi u glavnoj memoriji
- Tabele stranica mogu da budu u virtuelnoj memoriji
- Koristi se i pomoćna memorija za preslikavanje adresa (TLB) koja čuva do 32 unosa iz tabele stranica

V DINAMICKA ALOKACIJA MEMORIJE I SAKUPLJACI OTPADAKA

1. Rukovalac heap-om (heap manager) i dinamička alokacija memorije (pojednostavljen heap model)
   ➔ Heap menager
      - Gotovo svi moderni operativni sistemi nude mogućnost memorijske alokacije na zahtev
      - Rukovalac heap-om (heap manager) ima svrhu da omogući jednostavan mehanizam za dinamičko rukovanje memorijom
      - Proces od njega traži određen broj bajta ili oslobađa ranije tražene bajte:
         - void* malloc (byteSize);
         - void free();
      - Na ovaj način je moguće praviti dinamičke strukture podataka
      - Heap manager vodi evidenciju o zauzetim i slobodnim memorijskim segmentima
      - Funkcionalnosti heap manager-a
         - Alokacija memorije - preuzimanje slobodnog bloka iz liste slobodnih blokova, jednim od nekoliko algoritama
         - Dealokacija memorije - vraćanje bloka kako bi se uvrstio u listu slobodnih blokova.
         - Provera konzistentnosti heap-a
            • Vrši se najčešće pri dealociranju memorije
         - Funkcionalnosti ćemo objasniti na primeru, gde se
            • slobodni blokovi čuvaju u listi
            • Heap predstavlja niz kontinualnih memorijskih lokacija
   ➔ Pojednostavljen heap model
      - Alternativni naziv: Unmanaged memory heap manager
      - Brže je da svaki proces radi rukovanje autonomno nego da OS vodi računa o rukovanju memorije svakog procesa
      - Heap se može zamisliti kao niz bajta
      - Slobodni blokovi (npr. adresa i veličina) se čuvaju u listi slobodnih blokova
      - malloc() iz liste slobodnih blokova preuzima slobodan blok uz ažuriranje iste
      - free() blok koji se oslobađa uvezuje u listu uz ažuriranje liste
2. Alokacija memorije – algoritmi za alociranje. Dealociranje memorije.
   - OS održava listu slobodnih memorijskih blokova, sortiranu po adresi (rastući redosled)
   - Postoji više algoritama na osnovu kojih se alociranje može obaviti:
      - FirstFit
         - Po zahtevu lista se pretražuje od početka,
         - Memorija iz prvog kontinualnog segmenta iste ili veće dužine (u odnosu na zahtev) se dodeljuje aplikaciji
         - Preostali slobodni prostor, neposredno iza dodeljenog dela, vraća se u listu slobodne memorije
      - NextFit - varijacija first fit algoritma
         - Umesto da stalno polazi od početka, nastavlja od lokacije gde je zadnji put izvršena alokacija memorije. Neophodno je pamćenje te lokacije
      - Best Fit

        - OS pretražuje celu listu da nađe segment po veličini najsličniji traženom
    - WorstFit
        - Memoriju dodeljuje sa početka najvećeg raspoloživog segmenta
    - Last Fit
        - Čuva se lista vraćene memorije, po redosledu vraćanja
        - Favorizuje se dodela memorije iz najsvežijeg vraćenog segmenta (most recently released)
        - Pretpostavlja se da je taj segment još u skrivenoj memoriji
  - Pri alokaciji segment označava kontinualni niz slobodnih blokova (N blokova u nizu)

➔ Dealokacija memorije
  - Vraćanje memorije
  - Upis u listu slobodnih blokova
  - U zavisnosti od implementacije heap-a postoji više pristupa:
  - 1. Blokovi se vraćaju na početak liste, kako bi dealokacija bila što brža
  - 2. Blokovi se pri povratku sortiraju po adresi, što je sporije zbog pretrage i spajanja sa susednim blokovima. Eksterna fragmentacija je manja

3. Greške u radu sa dinamičkom memorijom – vrste grešaka, kratko objašnjenje i moguće rešenje problema.

➔ Curenje memorije
  - Situacija kada se u tekućem stanju programa izgubi informacija o lokaciji dinamički alociranog, a neoslobođenog bloka memorije
    char* p; p = (char*) malloc(1000);
    /* Deo koda u kome nema poziva free(p) */
    p = (char*) malloc(5000);
  - Program više nema mogućnost da oslobodi taj blok memorije i on biva ”zauvek“ (tj. do kraja izvršavanja programa) izgubljen
  - Curenja memorije koja se događaju u okviru neke petlje posebno opasna -> tada se gubi malo po malo memorije. Moguće je da u nekom trenutku program iscrpi svu raspoloživu memoriju i onda će njegovo izvršavanje biti prekinuto od strane OS
  - čak i da se to ne desi, moguće je da se iscrpi raspoloživi prostor u glavnoj memoriji i da se zbog toga sadržaj glavne memorije prebacuje na disk i obrnuto (tzv. swapping) - to veoma usporava rad programa
  - Većina programa za otkrivanje grešaka (debugger-a) detektuje curenje memorije u programu -> preostaje lociranje greške u kodu
  - Postoje specijalizovani programi profajleri za curenje memorije (eng. memory leaks profilers) koji olakšavaju otkrivanje uzroka curenja memorije

➔ Pristup oslobođenoj memoriji
  - Nakon poziva free(p), memorija na koju pokazuje pokazivač p se oslobađa, ali se ne menja sadržaj pokazivača p
    - Taj pokazivač se naziva dangling pointer („viseći pokazivač“)-> pokazivač koji ukazuje na memorijsku lokaciju koja je dealocirana tj. ne pokazuje na validni objekat

- Sistem može da realocira oslobođenu memoriju, pa bi u tom slučaju dereferenciranje pokazivača p izazvalo nepredvidivo ponašanje programa
- Zato je preporučljivo da nakon poziva free(p), resetujemo pokazivač na null vrednost:
  - p=NULL;
  - Tada će pokušaj pristupa oslobođenoj memoriji biti prepoznat -> OS će zaustaviti izvršavanje programa sa porukom o grešci (najčešće segmentation fault).

➔ Oslobađanje istog bloka više puta
- Nakon poziva free(p), svaki naredni poziv free(p) za istu vrednost pokazivača p prouzrokuje nedefinisano ponašanje programa
- Takozvana višestruka oslobađanja mogu da dovedu do pada programa

➔ Oslobađanje neispravnog pokazivača
- Prilikom poziva free(p), p mora biti adresa vraćena od strane funkcija malloc, calloc ili realloc
- Prosleđivanje pokazivača na lokaciju koja pripada alociranom bloku (ali nije njegov početak) uzrokuje probleme, kao na primer: free(p + 5)

➔ Prekoračenja i potkoračenja bafera
- pristup elementima van granica alociranog bloka može da prouzrokuje ozbiljne probleme u radu programa
  - Upis van granica bloka najčešće je opasniji od čitanja
- boundary overrun: upis iznad granice malloc regiona,
- boundary underrun: program piše pre početka alociranog bloka

4. Automatsko upravljanje memorijom – šta podrazumeva, šta je motivacija za njeno uvođenje i objašnjenje osnovnih pojmova (mutator, kolektor i koreni).
   - Motivacija
   - Terminologija
   - Problematika
   - Merenje kvaliteta
   - Algoritmi

➔ Motivacija
- Program se sastoji od komponenti koje između sebe komuniciraju definisanom spregom (Application Programming Interface, API)
- Prilikom prenosa argumenata između njih, ostaje otvoreno pitanje – ko je odgovoran za dealokaciju memorije (memory ownership)?
- U praksi se često ovo pitanje reguliše konvencijom – recimo komponenta koja alocira memoriju je dužna da je i dealocira što se može videti na API koji Windows pruža
- Ova problematika može dovesti do usložnjavanja dizajna aplikacije usložnjavanjem interfejsa, prenosa ownership-a i sličnog
- Ključni argument koji ide u prilog AMM je odvajanje problema rukovanja memorijom od dizajna aplikacije, a samim time i od problema koji aplikacija rešava
- Uz ovo, AMM može da reši ili smanji curenje memorije (memory leak), nevalidne pointere (dangling pointers), višestruko oslobađanje i fragmentaciju heap-a

- Jedan od argumenata zbog koji AMM postaje sve popularniji, a spominje se u industrijskim krugovima je – veći broj ljudi može da razvija aplikacije
- U praksi, ako posmatramo enterprise-grade aplikacije ovaj se argument pokazao upitnim, ako ne i netačnim. Određeno razumevanje rukovanja memorijom je neophodno za pravljenje kritičnih aplikacija

➔ Terminologija
- Mutator
  - Izvršava kod aplikacije koji alocira nove objekte i ažurira graf objekata ažuriranjem referenci na nove ili postojeće objekte
  - Dodaje kontrolne informacije (evidencija objekta)
  - Podržava operacije New(), Read() i Write()
- Kolektor
  - Detektuje objekte koji nisu dostupni, skuplja otpatke i oslobađa memoriju
- Koreni (GC Root)
  - Skup početnih referenci koji se nalaze u registrima, na steku ili globalne promenjive
  - Za svaki objekat koji je smešten u heap-u postoji jedan koreni pokazivač koji pokazuje na koren tog objekta

5. Sakupljač otpadaka (garbage collector, GC) – šta mu je zadatak i koji su osnovni tipovi GC?
   - čestim izvršavanjem dinamičkih alokacija javlja se „otpad“ pa je posao sakupljanja otpadaka (garbage collection) poveren delu sistema za upravljanje memorijom koji se naziva sakupljač otpadaka (eng. garbage collector, GC)
     - -> njegov zadatak je da sve ćelije koje nisu aktivne pretvori u slobodne
   - aktivni su svi objekti do kojih se može stići polazeći od svih postojećih korenih pokazivača
     - Na ovaj način sakupljač otpadaka je u stanju da odredi aktivne objekte i da zatim sve ostale proglasi slobodnim
   - Podela sakupljača na tri grupe:
   - 1. sakupljači otpadaka sa brojanjem referenci (reference counting garbage collectors)
   - 2. markirajući sakupljači otpadaka (mark-scan garbage collectors)
   - 3. kopirajući sakupljači otpadaka (copying garbage collectors)
6. Sakupljač otpadaka sa brojanjem referenci – objašnjenje principa rada. Prednosti i mane.
   - Uz svaki objekat se čuva broj referenci na njega
   - Broj referenci se ažurira u toku rada aplikacije
   - Kada broj referenci padne na nulu –> memoriju koju objekat zauzima je moguće osloboditi
   - U suštini, nije potrebno poznavati korene aplikativnog programa
   - Svaka ćelija u heap-u sadrži jednu reč koja se naziva brojač referenci
   - Sadržaj te reči je broj referenci na ćeliju, tj. broj pokazivača koji na nju pokazuju
   - Kada se neki podatak smesti u ćeliju i ona postane aktivna, njen brojač referenci dobija vrednost 1
   - Ako se kreira još neki pokazivač na tu ćeliju, njen brojač se povećava za jedan

- Ukoliko se uništi neki pokazivač na ćeliju, vrednost brojača referenci se smanjuje za jedan
- Ako vrednost brojača postane nula, to znači da ne postoji nijedan pokazivač na tu ćeliju-> ne može joj se pristupiti polazeći od nekog korenog pokazivača -> ona više nije aktivna
- Čim ćelija postane neaktivna, reciklira se i to tako što se povezuje u listu slobodnih ćelija
  - Izbegnuti su dugotrajni procesi traženja aktivnih ćelija i recikliranja neaktivnih, koji su karakteristični za ostale tipove sakupljača otpadaka
- recikliranje otpadaka obavlja se paralelno sa izvršavanjem programa, pa je ovaj tip sakupljača pogodan za real-time sisteme

➔ Mane
- Neefikasno jer se previše troši procesorsko vreme
  - Svaki put kada se pokazivač na neku ćeliju kreira ili uništi, njen brojač referenci mora da ažurira svoju vrednost
  - Ukoliko se vrednost nekog pokazivača promeni, moraju se ažurirati brojači dve ćelije – one na koju je pokazivač pokazivao pre promene svoje vrednosti i one na koju pokazuje posle promene
  - Dodatni posao stvaraju i pokazivači kratkog životnog veka, kao što su npr. oni koji se pojavljuju na steku prilikom prenošenja nekog objekta kao argumenta pri pozivu neke procedure
- zbog potrebe za brojanjem referenci neophodni dodatni biti (utrošak memorije) u odnosu na ostalie tipove sakupljača
- nesposobnost da reciklira ciklične strukture
  - u cikličnoj strukturi svaka ćelija ima bar jednu referencu, dok cela struktura može biti izolovana i ne postoji koreni pokazivač koji na nju pokazuje
- Ukoliko brojač referenci ćelije koja predstavlja koren neke drvoidne strukture dobije vrednost 0, onda će se reciklirati sve ćelije koje pripadaju toj drvoidnoj strukturi
  - Ako tu strukturu čini veći broj ćelija onda proces njihovog povezivanja u listu slobodnih ćelija može trajati duže tako da se oseti zastoj u radu programa -> nepoželjno u real-time sistemima

➔ Prednosti
- ne gubi mnogo od efikasnosti u slučaju kad je heap u velikoj meri ispunjen aktivnim ćelijama
- on je inkrementalne prirode:
  - ne moraju da se recikliraju sve neaktivne ćelije odjednom, već može deo po deo heap-a, trošeći po malo procesorskog vremena
  - ovo omogućava primenu u real-time sistemima
- većinu otpadaka reciklira neposredno nakon stvaranja
- Sakupljač otpadaka sa brojanjem referenci je našao primenu u Interlisp-u i ranijim verzijama Smalltalk-a

7. Markirajući sakupljač otpadaka (mark sweep) – kako funkcioniše, objašnjenje obe faze (faze markiranja i faze brisanja) i karakteristike

- Spada u grupu sakupljača otpadaka čije pronalaženje otpadaka se bazira na praćenju dostupnosti objekata na heap-u (eng. tracing garbage collector)
- Pretraga kreće počev od korenih pokazivača, pa se dalje prati niz referenci do objekata
- Objekat je dostupan ako je referenciran od strane korenih pokazivača ili referenciran od strane drugog objekta koji je dostupan
- oni objekti na koje ne postoje reference iz skupa korenih pokazivača, bilo direktne ili indirektne reference, su otpaci čija memorija se može osloboditi
- najstariji tip sakupljača otpadaka razvijen 1960. od strane John McCarthy u okviru programskog jezika LISP
- Sakupljanje otpadaka ima 2 faze:

➔ 1. faza: mark (faza obeležavanja) - obuhvata pronalaženje i markiranje aktivnih ćelija
  - Pod markiranjem se podrazumeva upisivanje jedinice u određeni bit (bit za markiranje) koji je pridružen svakoj ćeliji. Taj bit se nalazi ili u samoj ćeliji ili u bit-mapi
  - postupak markiranja u stvari predstavlja kretanje po dubini kroz grafovsku strukturu, počevši od skupa korenih pokazivača
  - Na kraju ovog procesa su markirane sve ćelije koje su naslednice korena
- 1. mark faza - Postupak markiranja:
  - kreće se od nekog korenog pokazivača i markira se ćelija na koju on pokazuje (mark bit = 1)
  - Ako markirana ćelija sadrži elementarni podatak, proces se završava
  - Ako markirana ćelija sadrži pokazivače, proces markiranja se primenjuje rekurzivno na ćelije na koje pokazuju ti pokazivači
- postupak se ponavlja za svaki postojeći koreni pokazivač
- na ovaj način sve aktivne ćelije u heap-u biće markirane

➔ 2. faza – sweep faza (faza brisanja)
- Heap se sekvencijalno pretražuje pri čemu se ćelije koje nisu markirane, a to su slobodne ćelije i otpaci, povezuju u listu slobodnih ćelija -> oslobađa se njihova memorija
- u ovoj fazi se bit za markiranje u svim aktivnim ćelijama postavlja na 0 kako bi se pripremili za naredni ciklus algoritma

➔ Karakteristike mark-sweep algoritma
- Lako se implementira
- U stanju je da reciklira i ciklične strukture, što je naročito bitno kod implementacije redukcije grafa
- Nedostaci:
- kada se aktivira proces sakupljanja otpadaka mora da se zaustavi program (Mutator) koji se izvršava
  - Ovo može izazvati da se program periodično „zamrzne“ što je problematično u slučaju real-time i aplikacija gde je vreme izvršavanja kritično
- Proces markiranja je rekurzivan -> sporo se izvršava

- Sve aktivne ćelije se posećuju dva puta (i u prvoj i u drugoj fazi), a neaktivne jednom (u drugoj fazi)
    - -> brzina sakupljanja otpadaka ne zavisi samo od broja aktivnih ćelija već i od celokupne veličine heap-a
- Moguća fragmentacija memorije jer se ne radi kompaktovanje
- Često se koristi kao osnova za druge algoritme

8. Trobojno markiranje – opis i objašnjenje algoritma
    - U mnogim implementacijama trobojnog markiranja na početku je crni skup prazan, sivi skup obuhvata objekte direktno dostupne preko korena, a svi ostali objekti su u belom skupu
    - Algoritam se vrši na sledeći način:
        - 1. Objekat X iz sivog skupa prebaci u crni skup
        - 2. Sve reference objekta X iz belog skupa premesti u sivi skup • Objekat X i sve njegove reference neće biti reciklirane
        - 3. Ponavljaj korake 1-2 dok sivi skup ne postane prazan
    - Po završetku algoritma:
        - sivi skup je prazan,
        - u crnom skupu su svi objekti koji su dostupni od korenih pokazivača
        - u belom skupu su otpaci koji se mogu reciklirati
    - Algoritam se drži pravila „nijedan objekat iz crnog skupa ne referencira objekat belog skupa“
    - Ovo obezbeđuje da se beli objekti mogu reciklirati čim se isprazni sivi skup
    - Prednosti algoritma:
        - Može da se izvodi u letu – bez zaustavljanja programa
          • Postignuto markiranjem objekata čim se alociraju i održavanjem više skupova objekata
        - Sakupljanje objekata se može vršiti periodično
9. Problem fragmentacije i kompaktovanje memorije
    - Problem fragmentacije:
        - Nema dovoljno memorije pri alociranju većih objekata
        - Alokacija postaje teža i neefikasnija
        - Duža je faza brisanja
        - Gubi se na lokalnosti – objekti alocirani zajedno imaju težnju da im se pristupa zajedno. Mešanjem novih alociranih objekata sa „starim“ narušava se lokalnost
    - Algoritmi kompaktovanja memorije rešavaju ove probleme premeštajući sve dostupne objekte u jedan kompaktan deo heap-a
    - Uopšteni zadatak kompaktovanja:
        - Premestiti dostupne objekte u jednu oblast heap-a
        - Izmeniti pokazivače tako da referenciraju objekte na novim lokcijama
    - Redosled objekata pri kompaktovanju može biti :

    - Proizvoljan (Arbitrary) - jednostavno, ali narušava raspored objekata na heap-u i prostornu lokalnost
    - Linearizujući (Linearizing) - grupiše objekte koji međusobno imaju reference
    - Klizajući (Sliding) - čuva trenutni raspored objekata na heap-u (lokalnost) -> najčešće korišćen u modernim kolektorima

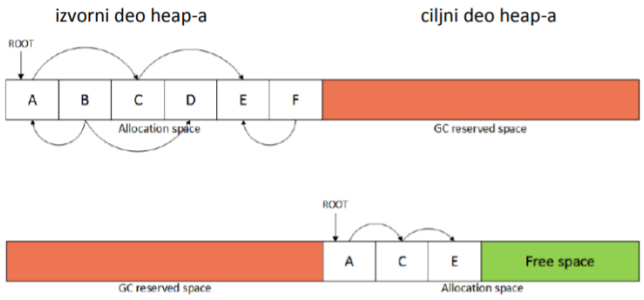
10. Mark compact algoritam – njegove faze i algoritmi koji ga predstavljaju:
    - Algoritmi se sastoje od tri faze:
      - Označavanje (mark),
      - Kompaktovanje,
      - Ažuriranje referenci

    a. Two-finger kompaktovanje – opis obe faze i objašnjenje
    - Osmišljen od strane Edvarda Sauncers-a 1974,
    - Koristi se za kompaktovanje objekata fiksne veličine, inače ne rezultuje punom defragmentacijom
    - Ovaj algoritam koristi proizvoljni redosled objekata nakon kompaktovanja
    - I. Faza kompaktovanja:
      - 1. Sastoji se od kretanja dva pokazivača jednog (free) sa početka heap-a i drugog (scan) sa kraja heap-a,
      - 2. Prvi pokazivač se kreće dok ne nađe slobodan prostor, a drugi smanjuje dok se ne nađe zauzet prostor,
      - 3. Objekat na koji pokazuje scan se kopira na lokaciju na koju pokazuje pokazivač free, a na staru lokaciju objekta se upisuje njegova nova adresa (pokazivač na novu lokaciju),
      - 4. Kada se free i scan pokazivači poklope – proces kompaktovanja je završen
    - II. Faza ažuriranja referenci:
      - 1. Proći kroz sve reference na objekte smeštene na heap-u,
      - 2. Ukoliko referenca pokazuje iznad vrednosti free pokazivača, koju on ima nakon faze kompaktovanja, onda:
      - 3. Referencu ažurirati novom vrednošću zabeleženom na staroj lokaciji objekta.
      - Nakon ažuriranja svih referenci, oslobađa se sva memorija iznad pokazivača free

    b. Lisp 2 kompaktovanje – opis faza i objašnjenje
    - klizajući alogoritam kompaktovanja
    - Dodaje se polje u zaglavlju svakog objekta za označavanje prosleđujuće adrese -> to je overhead i mana ovog algoritma
    - Koristi se na objektima različite veličine
    - Sastoji se od tri prolaza kroz heap:
    - 1. 1. Računanje nove adrese gde će se svaki dostupni objekat naći nakon kompaktovanja i 1.2. čuvanja te adrese unutar svakog objekta u polju forwardingAddress
    - 2. Ažuriranje referenci korena i markiranih objekata na vrednosti forwardingAddress
    - 3. Pomeranje objekata na izračunate nove lokacije

- Smer prolaska kroz heap (na gore) pri traženju novih lokacija je suprotan smeru premeštanja objekata (na dole)
- Ovo garantuje da kada se objekat pomera (u 3. prolazu algoritma) ta lokacija je već prazna
- dva pokazivača (free i live) kreću sa dna heap-a
- aktivni objekti se premeštaju na slobodno mesto zadržavajući originalni redosled

11. Kopirajući tip sakupljača otpadaka – objašnjenje na primeru dvoprostornog sakupljača otpadaka. Prednosti i mane.
    - 3.1 Dvoprostorni sakupljač otpadaka
    - Heap je podeljen na 2 dela:
        - 1. Izvorni deo- služi za smeštanje podataka nastalih u toku rada programa
        - 2. Ciljni deo– koristi se samo u toku rada sakupljača otpadaka – u njega se prepisuje sadržaj aktivnih objekata iz izvornog dela
    - Uslov: Veličina ciljnog dela mora biti ista kao veličina izvornog dela
    - Na kraju procesa sakupljanja, izvorni i ciljni deo heap-a zameniće uloge, tj. ciljni deo postaće izvorni i obrnuto



nakon skupljanja otpadaka: izvorni i ciljni deo heap-a zamenjuju mesta

    - Posao se obavlja u dve faze:
    - 1. Pronalaženje aktivnih ćelija u izvornom delu heap-a, po metodi mark sweep alogoritma (rekurzivno markiranje)
    - Pretraga aktivnih ćelija kreće od korenih pokazivača
    - Svaka aktivna ćelija se kopira u prvu slobodnu ćeliju ciljnog dela heap-a
    - premeštene ćelije formiraju kontinualan blok, čime se eliminiše pojava fragmentacije -> ovaj postupak naziva se i sažimanje, kompakcija
    - 2. Ažuriranje pokazivača
    - Nakon premeštanja ćelije, u njenu staru lokaciju upisuje se adresa nove lokacije
        - ta adresa se naziva adresa prosleđivanja (forwarding adress)
    - Prednosti:
        - Svaka aktivna ćelija posećuje se dva puta (jednom u prvoj i jednom u drugoj fazi), ali se neaktivne ćelije ne posećuju
        - -> brzina rada sakupljača otpada zavisi samo od broja aktivnih ćelija
        - -> brži od markirajućeg sakupljača
        - Nema fragmentacije memorije i lokalnost referenci je dobra

    - Jednostavan za implementaciju
  - Mane:
    - Proces sakupljanja otpadaka je rekurzivan
    - Iskorišćenost heap-a je najviše 50%, (ciljni deo se ne koristi za smeštanje podataka u toku izvršavanja programa)
    - mutator mora da se zaustavi i sačeka da sakupljač otpadaka završi svoj posao

12. Generacijski sakupljač otpadaka – šta je osnovna ideja, kako je organizovana memorija, kako i kad se vrši sakupljanje, šta je politika unapređivanja?
    - Kopirajući sakupljač otpadaka u svakom ciklusu sakupljanja otpadaka premešta sve aktivne objekte iz izvornog dela heap-a
    - To znači da će objekti koji imaju duži životni vek biti premeštani više puta u toku više ciklusa sakupljanja
    - Ideja je da se heap podeli na dva dela:
      - U jednom delu će se smeštati objekti dužeg životnog veka -> u njemu će biti manje otpadaka -> proces sakupljanja je ređi
      - Drugi deo će sadržati objekte kraćeg životnog veka -> tu će biti veći broj otpadaka-> proces sakupljanja će se češće vršiti
    - Kako odrediti koji će se objekti smeštati u koji deo heap-a?
    - većina skoro kreiranih objekata predstavlja privremene strukture ili međurezultate -> ti objekti imaju kratak životni vek
    - mali broj objekata je deo neke strukture dugog životnog veka
    - jednostavni kriterijum za razmeštanje objekata u heap-u:
      - tek kreirani objekti će biti smešteni u prvi deo heap-a koji će se češće sažimati
      - Oni objekti koji prežive određeno vreme biće iz tog dela premešteni u drugi u kome će se ređe vršiti sakupljanje otpadaka
    - Premeštanje objekta iz mlađe generacije u stariju naziva se unapređivanje ili promovisanje (promoting) objekata
    - Strategija koja određuje kada će se objekat unaprediti naziva se politika unapređivanja
    - Organizacija memorije:
    - Heap je podeljen na dva ili više regiona, pri čemu svaki region sadrži objekte približno iste starosti tj. iste „generacije“
    - Svaki region se označava brojem koji predstavlja oznaku generacije objekata koje on sadrži
      - Region 1 – sadrži najmlađu generaciju objekata
      - Region n – sadrži najstariju generaciju objekata
    - veličina regiona se smanjuje sa povećanjem generacijskog broja
    - Veličina regiona 1 – tu se smeštaju novi objekti -> naziva se i oblast smeštanja
      - Treba da je dovoljno veliki tako da se sporije popunjava i tada se kompakcija ređe vrši
    - Implementacija sakupljanja
      - vrši se kopirajućim algoritmom

    - u regionima koji sadrže mlađe generacije, gde je procenat živih objekata mali, efikasna je dvoprostorna varijanta kopirajućeg algoritma
  - Politika sakupljanja otpadaka:
    - Određuje koliko često će se otpaci sakupljati po regionima
    - Najčešće se region sažima kada se popuni objektima
  - Politika unapređivnja:
    - Određuje kada će se objekti premestiti u stariju generaciju
    - Najjednostavnija i najčešće primenjivana politika unapređivanja je ona koja unapređuje objekte koji prežive jednu kompakciju regiona
13. CLR heap manager – kriterijumi po kojima je heap particionisan. Koliko generacija objekata koristi GC i šta je u kojoj smešteno?
  - Heap je particionisan po nekoliko kriterijuma, po:
    - životnom ciklusu objekta: 3 generacije - Gen 0, Gen 1 i Gen 2
    - veličini objekta: SOH (small object heap) i LOH (large object heap)
      - SOH služi za smeštanje objekata manjih od 85000 bajta
      - LOH služi za smeštanje objekata većih od 85000 bajta
  - Sakupljanje memorije se sastoji od sledećih faza:
    - Mark,
    - Plan,
    - Relocate,
    - Compact,
    - Sweep
  - .NET framework koristi GC koji svrstava objekte u 3 generacije:
    - Genearcija 0, generacija 1 i generacija 2
  - Generacija 0 – tu se smeštaju novi, sveže alocirani objekti
    - Tu su objekti koji uglavnom kratko žive
    - Skupljanje smeća se najčešće dešava u ovoj genearciji
    - Ako se alociraju veliki objekti oni se stavljaju na LOH (Large Object Heap) – koji kao logička celina pripada generaciji 2
    - Većina objekata iz Gen. 0 ne preživi do sledeće generacije i bivaju pokupljeni od strane GC
  - Generacija 1 – tu su objekti koji su preživeli jedan ciklus kolekcije
    - Ova generacija služi kao prelaz između objekata kratkog životnog veka (Gen. 0) i objekata dugog životnog veka (Gen. 2)
    - Nakon kolekcije unutar Gen. 0, GC kompaktuje memoriju sa aktivnim objektima i promoviše ih u Gen. 1
  - Generacija 2 – sadrži objekte koji su preživeli više od jednog GC ciklusa
    - Objekti ostaju u Gen. 2 dok ne postanu nedostupni u nekoj budućoj kolekciji
    - Objekti na LOH se takođe kolektuju unutar generacije 2
14. Oslobađanje memorije i promovisanje objekata u .NET framework-u.

➔ Oslobadjanje memorije

- GC oslobađa memoriju postupno po generacijama umesto oslobađanja u čitavom heap-u
- Skupljanje otpadaka u generaciji označava da se objekti skupljaju u toj generaciji i svim mlađim generacijama
- Ako aplikacija pokuša da kreira novi objekat, a generacija 0 na heapu je popunjena -> tada GC pokreće skupljanje otpadaka u generaciji 0 da bi oslobodio memorijski prostor
  - Prelazi se preko grafa objekata i markiraju se živi objekti, a oslobađaju objekti koji nisu markirani
  - objekti koji su markirani promovišu se u sledeću generaciju -> u generaciju 1
- kolektovanje otpadaka u generaciji 0 često oslobodi dovoljno memorije da aplikacija može da nastavi sa radom i novim alokacijama
  - Ali ako nakon kolektovanja u generaciji 0 i dalje nema dovoljno slobodne memorije -> vrši se ciklus kolekcije nad generacijom 1
- Objekti koji prežive kolekciju u generaciji 1 prebacuju se u sledeću generaciju tj. generaciju 2
- U slučaju kolektovanja generacije 2, preživeli objekti ostaju u toj generaciji, brišu se samo mrtvi objekti

➔ Promovisanje objekata
- Objekti koji prežive koleciju nad jednom generacijom objekata promovišu se u sledeću višu generaciju
- Kada sakupljač smeća otkrije da je stopa preživljavanja visoka u generaciji, povećava prag izdvajanja memorije za tu generaciju
- CLR stalno balansira između dva prioriteta: ne dozvoliti da heap aplikacije postane prevelik tako da se sakupljanje otpadaka odlaže, ali i ne dozvoliti da se sakupljanje smeća izvodi prečesto zbog premaloga heap-a aplikacije

VI KOMUNIKACIONO ORJENTISANI SLOJEVI ISO OSI MODELA

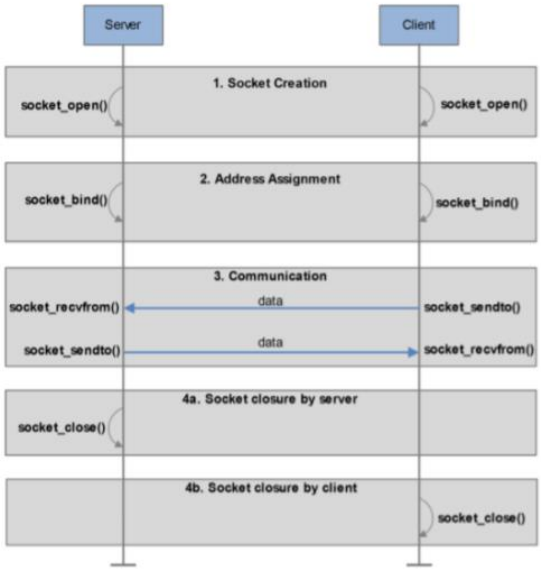
1. TCP/UDP funkcionalnost i TCP/UDP adresiranje. Pojam soketa (utičnice).

➔ TCP/UDP funkcionalnost
- TCP: Connection-oriented, stream, reliable, error-free delivery
  - TCP paketi su označeni brojem sekvence, koji se ponavlja ali u dužem periodu
  - Isporuka se organizuje po tom broju
  - U slučaju greške, TCP radi retransmisiju paketa
- UDP: connectionless, message, unreliable datagram service
  - Podrazumeva kontrolu isporuke na aplikacionom nivou
  - Jednostavniji i efikasniji
  - Manja kašnjenja pogoduju multimedia aplikacijama
  - Npr, izgubljeni VoIP paket se ne zanavlja
- Oba protokola su FDX, klijent-server usluga

➔ TCP/UDP adresiranje
- Socket = IP adresa + broj porta (prolaza)

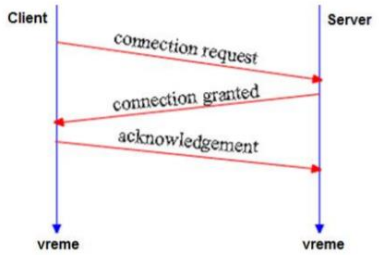
        - Adresa (npr. 192.52.1.33) identifikuje računar
        - Port je broj n[1 - 65535] dodeljen korisničkoj aplikaciji
        - Prvih 1023 “portova” su rezervisani
    - Poruke se pre slanja dele na pakete
        - Dodaje im se TCP/UDP zaglavlje
    - Na obe strane se formiraju Tx/Rx baferi
        - Kontrola toka sprečava prepisivanje/prepunjavanje na prijemu
➔ Soket (socket, uticnica)
    - Preko soketa procesi šalju poruke u mrežu
    - Standardna API sprega (Application Programming Interface)
    - Multipleksiranje / demultipleksiranje poruka

2. Osnovne karakterisitke UDP protokola. Aplikacije koje ga koriste.
    - Protokol za razmenu korisničkih datagrama
    - Message-oriented – isporučuje poruke
    - U suštini UDP proširuje IP uvođenjem koncepta prolaza (eng. port)
        - UDP = IP + adresiranje aplikacije
        - Nema uspostave sesije i
        - Ne garantuje isporuku paketa
        - “best effort” usluga
    - Pošto je UDP jednostavniji od TCP-a, on je i brži.
    - UDP server je definisan parom
        - [IP adresa, port]
        - Svaki datagram sa tim odredištem biće prihvaćen, bez obzira ko ga šalje
    - Prenos multimedijalnih sadržaja
        - loss tolerant
        - rate sensitive
    - Internet UDP korisnici
        - DHCP
        - DNS
        - SNMP
    - Aplikacije koje koriste UDP moraju same obezbediti kontrolu i oporavak u slučaju grešaka
3. Tok aplikacije u slučaju UDP utičnica.
    - Open – evidentiranje kod UDP drivera
    - Bind – omogućuje prijem, definiše servera
    - SendTo/ RecvFrom - nezavisna razmena paketa podataka, druga strana se specificira parom Adresa / Port
    - Close – obe strane mogu ga inicirati
    - App poziva funkcije UDP rukovaoca, koji je deo OS



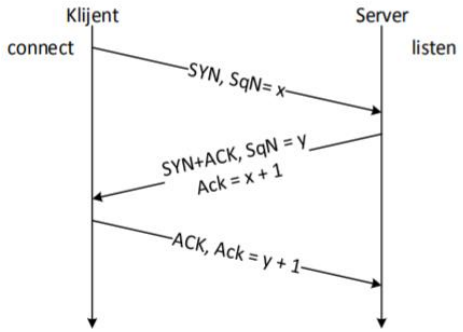
4. Osnovne karakterisitke TCP protokola. TCP logički tok.
   - TCP je protokol pouzdanog prenosa koji se nalazi u transportnom sloju.
   - Radi na vrhu IP koji je deo mrežnog sloja.
   - Kako je IP nepouzdan protokol, TCP realizuje mehanizme kontrole toka i prenosa koji obezbeđuju pouzdan prenos informacije.
   - TCP potvrđuje svaki paket koji primi, tako da pošiljaoc zna da je paket stigao na svoje odredište.
   - U slučaju da TCP sa predajne strane ne primi potvrdu, on ponovo šalje nepotvrđeni paket (eng. retransmission).
   - Na taj način aplikacija može računati da će mreža obaviti pouzdanu isporuku informacije željenom odredištu.

   ➔ TCP logicki tok
   - TCP je stream-oriented, jer sve podatke u prenosu tretira prosto kao niz okteta.
   - Isporuka na odredištu jeste identična po broju i redosledu okteta, ali ne i po porukama na ulazu u socket
5. TCP sesija – uspostava sesije i prekid sesije.
   - Definisana je sa dva para [IP adresa, port], za procese na oba kraja konekcije
   - Sesija za svaki par računara u TCP vezi
   - Uspostava prethodi razmeni paketa, 3 koraka

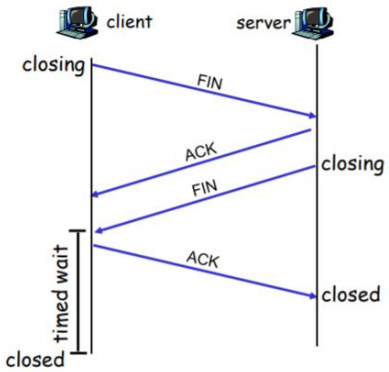


   ➔ Uspostava sesije

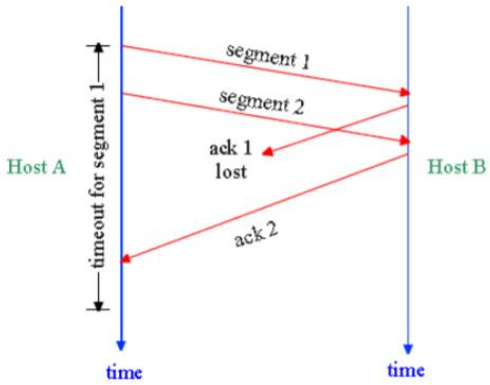
- Klijent inicira, Server prihvata konekciju
- Razmena inicijalnih (random) vrednosti brojača sekvence



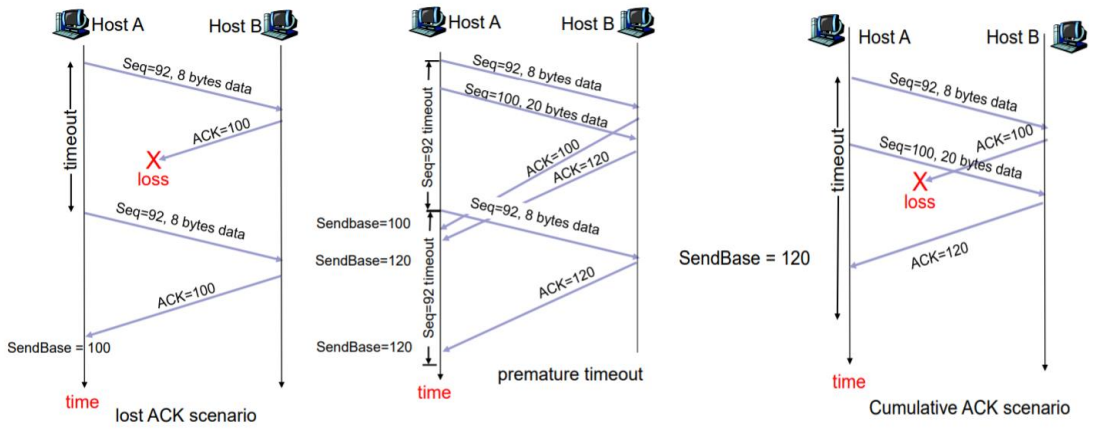
➔ Prekid sesije
- Obe strane mogu prekinuti sesiju, ali to obično radi klijent
  - Active / pasive close
- Razmenjuju se FIN i ACK paketi



6. Potvrda paketa i scenariji zanavljanja kod TCP protokola.
   - Pri slanju segmenta se startuje timer i čeka potvrda
   - Ako potvrda izostane vrši se retransmisija
   - Koristi mehanizam klizajućeg prozora



➔ Scenariji zanavljanja



7. Osnovni zadaci i karakteristike IP protokola.
    - IP je zadužen za prenos i usmeravanje paketa
        - Internet Protocol
    - IP paketi (datagrami) se prosleđuju preko fizičkog nivoa
    - IP je mrežni protokol
        - internetworking – povezivanje heterogenih udaljenih mreža
        - slanje i rutiranje datagrama
    - Rukuje slanjem paketa kroz mreže
        - Koristi adresu iz zaglavlja paketa, i dodaje adresu pošiljaoca
        - IP datagram je osnovna jedinica prenosa u Internetu
        - Max dužina 64k, ipak 1500 zbog ograničenja kašnjenja i opterećenja mreže (čvorova)
        - best-effort service, tj. ne zanavlja izgubljene pakete
    - Zaglavlje IPv4 protokola
        - Najvažnije su IP adrese izvora i odredišta
        - Ostalo su dodatni elementi neophodni za prenos i rukovanje IP paketima
        - 20 bajta + opcije
    - IP Fragmentacija i Prikupljanje
        - MTU (max. transfer unit) je najduži okvir u prenosu
            - Karakteristika svake deonice
            - Menja se u toku prenosa
        - Veliki IP datagrami se dele (fragmentiraju) unutar mreže
            - Podela na manje pakete
            - Prikupljanje na odredištu
            - IP zaglavlje identifikuje ove pakete i njihov redosled
        - IPv6 ne dozvoljava fragmentaciju na mrežnom nivou
    - Zaglavlje IPv6 protokola
        - Ciljevi v4→v6 tranzicije
            - 1. Proširenje IP adrese
            - 2. Zaglavlje fiksne dužine radi brže obrade

- 3. Opcije su prebačene u podatke (next hdr ukazuje na tip i poziciju)
- 4. Inače ukazuje na gornji protokol
- 5. Nema cheksum-a
- 6. Protokol više ne podržava fragmentaciju
- 7. Podrška za QoS (pri, flow)

VII PARALELNO RACUNARSTVO

1. Paralelno računarstvo – pojam, tipovi paralelizma, poređenje konkurentnosti i paralelizma

➔ Pojam

- Paralelno računarstvo predstavlja istovremeno korišćenje više računarskih resursa u cilju rešavanja nekog problema
  - Softver se izvršava na više procesora
  - Problem se razdvaja na više delova koji se mogu rešavati uporedno (u paraleli)
  - Svaki deo problema se dalje razdvaja na serije diskretnih instrukcija
  - Instrukcije iz svakog od delova se izvršavaju istovremeno na različitim procesorima
- Pisanje paralelnih programa može biti veliki izazov, zahteva se podela problema i definisanje komunikacije i sinhronizacije. Znanje arhitekture i karakteristike računara je veoma važno.

➔ Tipovi

- Na nivou bita
  - Paralelizam preko veličine reči 8, 16, 32, ili 64 bita
- Na nivou instrukcija (ILP) • Pipelining
- Na nivou podataka
  - vector units, SIMD execution, …
  - increasing … SSE, AVX, Cell SPE, Clearspeed, GPU
- Na nivou niti
  - increasing … multithreading, multicore, manycore
  - Intel Core2, AMD Phenom, Sun Niagara, STI Cell, NVIDIA Fermi, …

➔ Konkurentnost vs. paralelizam

- Konkurentan rad na računaru
  - 1 ili više CPU-a
  - Kvazi-simultano izvršavanje
- Paralelan rad na računaru
  - Više CPU-a
  - Simultano izvršavanje mnogo/svih niti iste aplikacije
- Zajedničke osobine:
  - Niti/procesi se preklapano (isprepleteno) izvršavaju
  - Sinhronizacija, komunikacije
  - Sukob oko resursa, uslovi trke, potpuni zastoji
- Cilj konkurentnog izvršavanja:

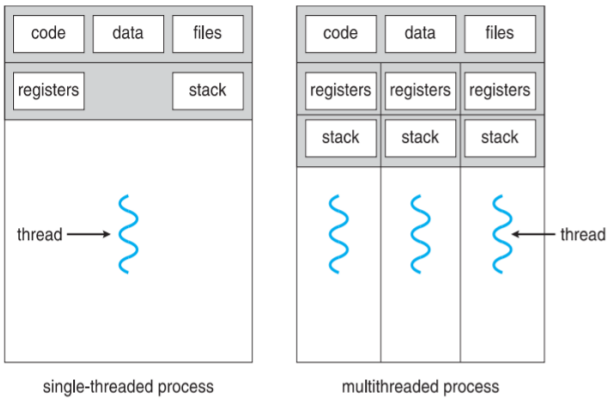
    - Povećanje CPU iskorišćenja
    - Povećanje brzine odziva sistema
    - Podrška rada sa više korisnika
  - Centralna pitanja za konkurentan rad:
    - Raspoređivanje, prioriteti, …
  - Cilj paralelnog izvršavanja:
    - Ubrzanje jedne aplikacije (veliki problem)
  - Centralna pitanja za paralelan rad :
    - Paralelni algoritmi i strukture podataka, mapiranje, balansiranje opterećenja
  - Paralelizam i konkurentnost su međusobno povezani ali:
    - moguć je paralelizam bez konkurentnosti (kao što je bit-level parallelism) i
    - konkurentnost bez paralelizma (kao što je multitasking sa time-sharing na CPU procesoru sa jednim jezgrom).

2. Pojam procesa i niti, procesi sa jednom ili više niti, harverske niti i softverske niti

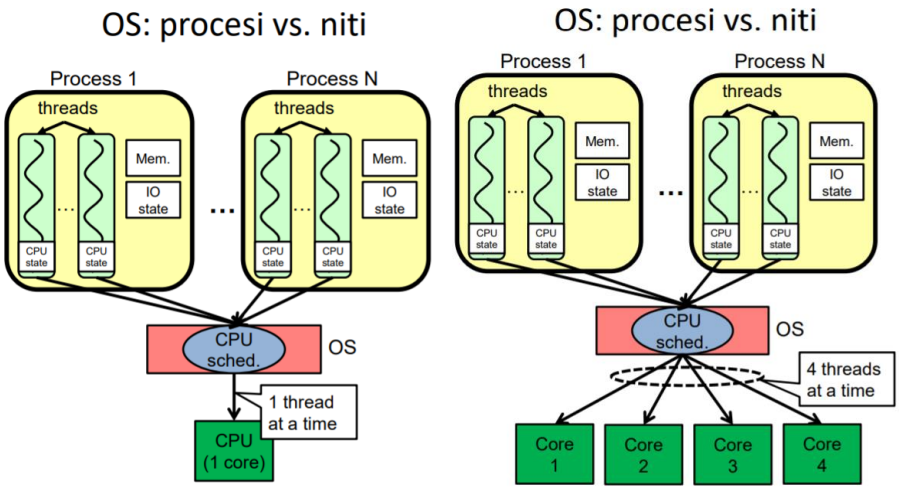
➔ Hardverske niti vs. softverske niti

- "Hardverska nit" je fizički CPU ili jezgro. Jedna hardverska nit može da startuje-izvršava mnogo softverskih niti. U modernim operativnim sistemima, ovo je često dato time-slicingom gde svaka nit dobija nekoliko milisekundi da se izvrši pre nego što operativni sistem rasporedi drugu nit za izvršenje na tom CPU
- Svako fizičko jezgro može da ponudi više od jedne hardverske niti (logičko jezgro ili logički procesor).
- Svaki program koji se izvršava u Windows operativnom sistemu je proces. Svaki proces kreira i startuje jednu ili više softverskih niti.

➔ Procesi sa jednom ili vise niti



➔ Procesi vs. niti

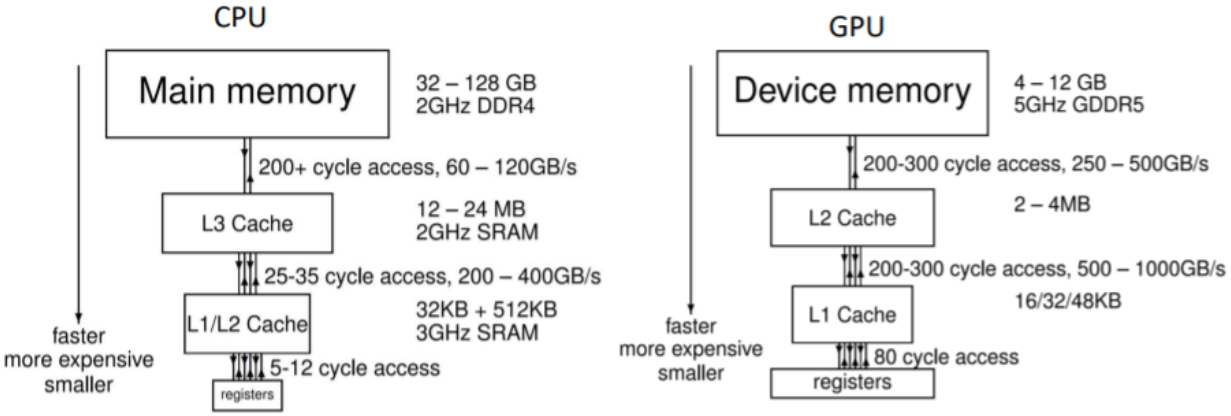


3. Poređenje karaketristika CPU i GPU procesora
    - Centralna procesorska jedinica - CPU
        - Optimizovana za pristup keširanom skupu podataka sa malim kašnjenjem
        - Kontrolna logika za neispravno izvršavanje
        - Izvršenje desetak niti
    - Grafička procesorska jedinica - GPU
        - Optimizovana za paralelan rad sa podacima i povećanje propusnosti
        - Arhitektura tolerantna na memorijska kašnjenja
        - Izvršavanje desetak hiljada niti
    - Različiti ciljevi proizvode različite dizajne
        - CPU mora biti efikasan za svaki problem, bilo da je paralelan ili ne
        - GPU pretpostavlja da je radno opterećenje visoko paralelno
    - CPU: minimizira kašnjenje dobijeno od jedne niti
        - Velike keš memorije na čipu
        - Sofisticirana kontrolna logika
    - GPU: maksimizira propusnost svih niti
        - Broj niti koje se izvršavaju je ograničen resursima => potrebno je dosta resursa (registri, propusnost, itd.)
        - multithreading može da sakrije kašnjenja => preskoči veliki keš
        - Deli kontrolnu logiku između više niti
    - CPU - task paralelizam
    - GPU - data paralelizam
    - Kompleksnost GPU-a je uveliko smanjena (nemaju inteligenciju CPU-a, nemaju prefetch, nemaju branch prediction …)
    - GPU procesori namenjuju najviše svojih tranzistora za procesiranje, sledi da su oni idealni za računarski-intenzivna, visoko paralelizovana izračunavanja
        - Efekti u video igrama, računarske naučne simulacije, procesiranje slike i mašinsko učenje, linearna algebra ...

CPU Xeon Platinum 8180	GPU NVIDIA V100
One of the largest Intel CPUs is the Xeon Platinum 8180. This has: •28 Cores •Runs at a frequency of 2.5 GHz •Costs ~ £7.000. ...	One of the largest NVIDIA GPUs is the V100. This has: •5120 Cores •Runs at a frequency of 1.37 GHz •Costs ~ £8.800. ...

Poređenjem Intel-CPU procesora Xeon Platinum 8180, i NVIDIA-GPU procesora V100, može da se vidi da su najbolje računarske performanse V100 oko 3 puta veće, a da je cena samo 25% veća, ali
CPU je fleksibilniji od GPU-a i može nadmašiti GPU za određene računarske probleme

4. Poređenje CPU i GPU memorijske hijerarhije

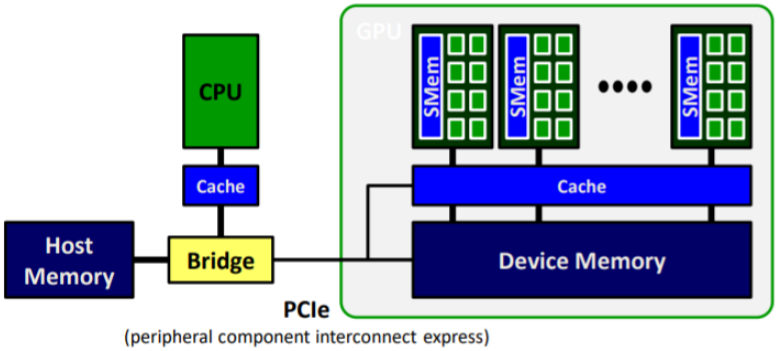


- Moderni računari mogu imati više različitih tipova memorije sa različitom propusnošću (količina podataka koja se može transferovati između dve tačke u jedinici vremena).
- Memorije se mogu urediti po hijerarhiji, od spore RAM memorije do veoma brzih registara koji su locirani blizu CPU jezgara za procesiranje.
- Mada GPU jedinice imaju veću propusnost one imaju manju keš memoriju i veća kašnjenjamemory latency (vreme transfera podataka od memorije do jezgara za procesiranje).

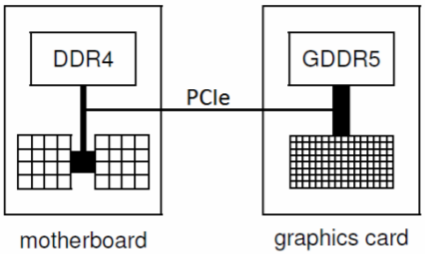
5. GPU sistem memorija
   - - Nit, blok niti, rešetka blokova niti
   - Globalna memorija: najsporija i najveća memorija na GPU, koju dele sve rešetke (skup blokova niti)
   - Deljena memorija: veoma brza memorija, locirana na SM-u (Streaming Multiprocessor), dele je niti bloka

- Registri: najbrža moguća memorija, locirana na SM-u, primena za niti
- Lokalna memorija: locirana na globalnoj memoriji, primena za niti
- L1/L2/L3 keš memorije – smanjuje kašnjenje pri pristupu memoriji
- Keš memorija za grafiku-teksture i konstantne varijble

6. Heterogeno računarstvo – pojam i tipična konfiguracija



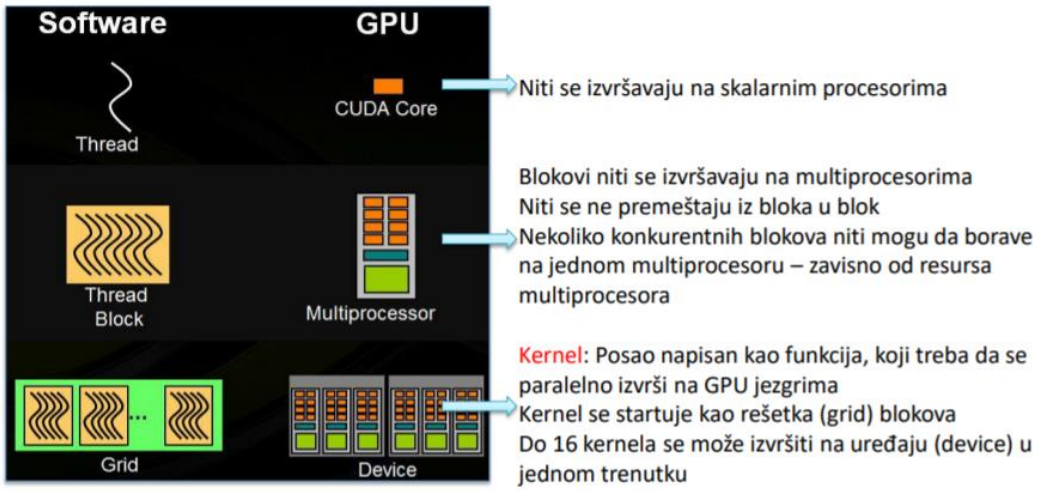
- Heterogeno računarstvo koristi više od jednog tipa procesora (npr. CPU i GPU). Naziva se i hibridno računarstvo ili "ubrzano" (accelerated computing). Razlozi : energetska efikasnost, veća propusnost ... Server se obično zove host, a GPU device Veza preko PCIe je diskretna a postoji i integrisana verzija CPU i GPU na jednom čipu



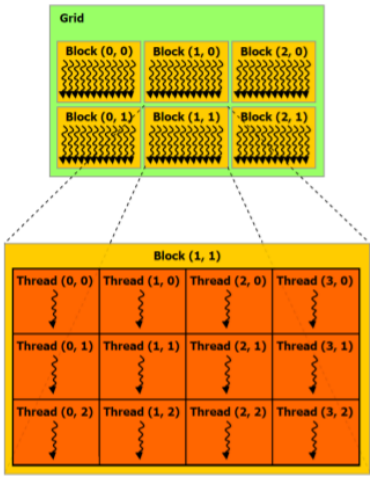
- Tipična serverska konfiguracija ima jedan ili više CPU-a (obično dva) koji komuniciraju sa jednim ili više grafičkih procesora preko PCIe busa.
- CPU: velike keš memorije, nekoliko niti po jezgru, umerena širina opsega memorije zasnovana uglavnom na keš memoriji i prefetching-u
- GPU: male keš memorije, veliki broj niti, velika širina opsega memorije zasnovana na multi-threading karakteristikama

7. GPU arhitektura – pojmovi: stream processor (SP), stream multiprocessors (SM) , SIMT (Single Instruction Multiple Thread), warp
   - Ključne ideje za arhitekturu:
     - SIMT(Single Instruction Multiple Thread) izvršenje
       - grupa 32 niti koje se izvršavaju zove se warp
       - niti u warp-u dele instruction unit (IU)
       - HW automatski rukuje divergencijom
     - Hardverski multithreading
       - hardverska alokacija resursa i raspoređevanje niti
       - hardver se oslanja na niti da bi sakrio kašnjenje (latency)
     - Niti imaju sve resurse potrebne za start izvršenja

        - bilo koja nit ne čeka ako može da se izvrši
        - promena konteksta (context switching) je u osnovi bez troškova
    - GPU jezgro se zove stream processor (SP). Stream processors su grupisani u Stream Multiprocessors (SM). SM su SIMD procesori (takođe se zovu Single Instruction Multiple Threads, jedna instrukcija primenjena na više jezgara - multiple cores).
    - Niti se na svakom SM-u izvršavaju u grupama od 32 niti koje se zovu warp-ovi. Niti u warpu dele IU (instruction unit)
    - Izvršavanje se alternativno smenjuje između "aktivnih" warp-ova i worp-ova koji postaju privremeno "neaktivni" kada čekaju na podatke
    - GPU ne koriste promenu konteksta (context switching); svaka nit ima sopstvene registre, koji limitiraju broj aktivnih niti
8. GPU i CUDA programski model (odnos niti, bloka i kernela naspram GPU jezgra, SM i GPU uređaja).



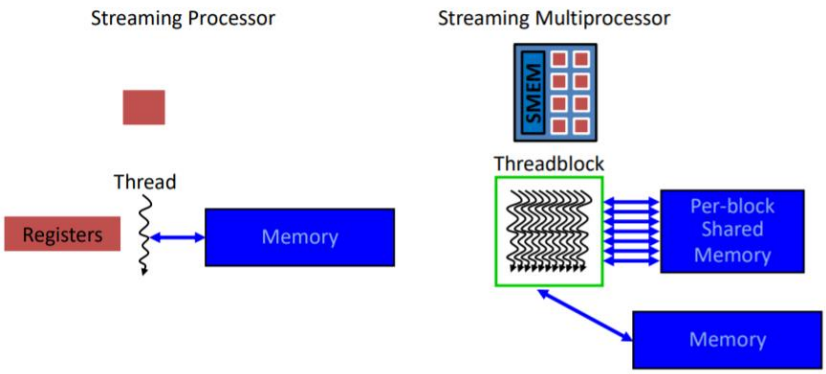
9. Organizacija niti u CUDA programu (nit, blok, rešetka). Dimenzije bloka i rešetke.
    - CUDA programi se sastoje od hijerarhije konkurentnih niti
    - Svaka nit u bloku ima jedinstvenu threadIdx vrednost
    - Svaki blok ima jedinstvenu blockIdx vrednost
    - dim3 je struktura (definisana u fajlu vector_types.h) koja definiše grid i block dimenzije
    - Radi se sa tri dimenzije:
        - dim3 grid(256); // definiše rešetku od 256 x 1 x 1 blokova
        - dim3 block(512, 512); // definiše blok od 512 x 512 x 1 niti
        - threadIdx i blockIdx su promenljive tipa dim3
        - foo<<>>(...);
            - matrixAdd<<>>(A, B, C);



dim3 threadsPerBlock(4, 3, 1) definiše blokove sa 4 x 3 x1 niti (slika iznad)

10. GPU struktura i njegova veza sa nitima, vorpovima, blokovima niti i rešetkom.
    - Svaki blok je dodeljen jednom SM-u
    - Unutar SM-a blok je podeljen na Warp-ove
        - Warp je sastavljen od 32 niti
            - CUDA definiše konstante u fajlu cuda_runtime.h
        - Svaka od 32 niti MORA da izvršava isti set instrukcija u isto vreme
            - Zbog činjenice da postoji samo jedna instrukcijska jedinica (instruction unit)
        - Warpovi se izvršavaju konkurentno na jednom SM-u
        - Ako kernel pokuša da ima niti koje rade različite stvari u jednom worpu (korišćenje if naredbe na primer), dva taska će se izvšavati sekvencijalno
    - Maksimalan broj niti u bloku je obično 512 ili 1024 zavisno od mašine
    - Maksimalan broj blokova u gridu je obično 65535
        - Ako se pređe bilo koja vrenost GPU će prestati da radi ili se prijavljuje greška u podacima
11. CUDA model paralelizma. Izvršavanje niti i blokova
    - Generički manycore čip
    - CUDA virtualizuje fizički hardver
        - Nit je virtualizovan skalarni procesor (registri, programski brojač - PC, stanje)
        - Blok virtualizuje multiprocesor (niti, deljena memorija)
    - Raspoređivanje na fizički hardver bez prekidanja (no-preemption)
        - niti/blokovi se startuju i izvršavaju do kompletiranja
        - Blokovi treba da budu nezavisni

Izvršavanje niti i blokova



12. Kreiranje paralelnog programa – faze i njihov opis
    - Identifikovanje rada koji se može izvršiti "paralelno"
    - Deljenje posla (a takođe i podataka pridruženih tom poslu)
    - Upravljanje pristupom podacima, komunikacijom i sinhronizacijom

➔ Dekompozicija
- Ko je odgovoran za dekompoziciju? U većini slučajeva: programer
- Amdalov zakon: ograničenja maksimalne brzine izvršavanja, povezano sa paralelizmom
- Ako je S sekvencijalno izvršenje, maksimalno ubrzanje (speedup) zbog paralelnog izvršenja je ≤ 1/S

➔ Dodela
- Dodela taskova nitima
  - Kada se kaže "tasks" misli se na stvari koje treba da se urade
  - O nitima treba razmišljati kao o radnicima
- Ciljevi: balansiranje radnih opterećenja, smanjenje komunikacionih troškova
- Može da se izvršava statički, ili dinamički
- U mnogi jezicima vremena izvršenja (runtimes) su odgovorna za dodelu

➔ Usklađivanje rada
- Uključuje:
  - Strukturiranje komunikacija
  - Dodavanje sinhronizacije da bi se sačuvale zavisnosti ako je to neophodno
  - Organizovanje struktura podataka u memoriji
  - Raspoređivanje taskova

➔ Mapiranje
- Mapiranje niti (radnika-workers) na hardversku jedinicu izvršenja

13. CUDA upravljanje memorijom – cudaMalloc(), cudaFree(), cudaMemcpy()
    - cudaMalloc(ptr, datasize)
    - cudaFree(ptr)
    - Inicijalizacija memorije sa cudaMemset()
    - Slično kao CPU upravljanje memorijom, ali dato da CPU alocira-dodeljuje memoriju GPU
    - cudaMemcpy(void *dst, void *src, size_t nbytes, enum cudaMemcpyKind direction);

- cudaMemcpy je sinhrona funkcija
    - Vraća kontrolu nakon što se kopiranje kompletira
    - Blokira CPU nit dok se svi bajtovi ne kopiraju
    - Ne počinje da kopira dok se prethodni CUDA poziv ne kompletira
- enum cudaMemcpyKind
    - cudaMemcpyHostToDevice
    - cudaMemcpyDeviceToHost
    - cudaMemcpyDeviceToDevice

14. CUDA programiranje – host kod i kernel kod
    - CUDA program je sastavljen iz dva dela:
        - 1. host kod koji se izvršava na CPU koji je povezan sa GPU
        - 2. Kernel kod koji se izvršava na GPU

CUDA programiranje – host kod

Na nivou host koda, postoje bibliotečke rutine za:

- alokacija memorije na grafičkoj kartici
- transfer podataka ka/od device memorije, uključujući
    - •konstantne
    - •obične podatke data
- error-checking
- timing

Postoji specijalna sintaksa za startovanje više instanci kernel procesa na GPU.

```
// Allocate pointers for host and device memory
float *h_input, *h_output;
float *d_input, *d_output;

// malloc() host memory (this is in your RAM)
h_input= (float*) malloc(mem_size);
h_output= (float*) malloc(mem_size);

// allocate device memory input and output arrays
cudaMalloc((void**)&d_input,  mem_size);
cudaMalloc((void**)&d_output, mem_size);

// Do something here!

// cleanup memory
free(h_input);
free(h_output);
cudaFree(d_input);
cudaFree(d_output);
```

```
// Copy host memory to device input array
cudaMemcpy(d_input, h_input, mem_size, cudaMemcpyHostToDevice);

// Do something on the GPU

// copy result from device to host
cudaMemcpy(h_output, d_output, mem_size,
cudaMemcpyDeviceToHost);
```

CUDA programiranje – host kod

U svojoj najprostijoj formi specijalna sintaksa izgleda kao:

kernel_routine<<<gridDim, blockDim>>> (args);

gridDim je broj instanci kernela (gridDim veličina rešetke)

blockDim je broj niti u svakoj instanci (blockDim veličina bloka)

Args je ograničen broj argumenata, obično uglavnom pokazuje na nizove u grafičkoj memoriji i na neke konstante koje se kopiraju po vrednosti

Opštija forma dozvoljava da gridDim i blockDim budu 2D ili 3D da bi pojednostavili program aplikacije

```
__global__ void helloworld_GPU(void){
  printf("Hello world!\n");
}

Int main(void) {

// run CUDA kernel
helloworld_GPU<<<1,1>>>();
?
return(0);
}
```

Kernel kod

```
#include <helper_cuda.h>

__global__ void my_first_kernel(float *x)
{
Int tid= threadIdx.x+ blockDim.x*blockIdx.x;
x[tid]  = (float) threadIdx.x;
}
```

- __global__ identifikator kaže to je kernel funkcija
- Svaka nit postavlja jedan element x niza.
- Unutar svakog bloka niti, threadIdx.x uzima vrednosti iz opsega od 0 do blockDim.x-1, tako da svaka nit ima jedinstvenu vrednost *tid*.

15. Tipičan tok CUDA programa
    1. Load data into CPU memory
       fread/rand
    2. Copy data from CPU to GPU memory
       cudaMemcpy(..., cudaMemcpyHostToDevice)
    3. Call GPU kernel
       yourkernel<<>>(...)
    4. Copy results from GPU to CPU memory.
       cudaMemcpy(..., cudaMemcpyDeviceToHost)
    5. Use results on CPU

16. Kerneli – šta su, kako se pokreću (launch kernel), objašnjenje parametara na primeru poziva jednog kernela: KernelName<<>>(arg1, arg2, ...)
    - Specijalne funkcije koje CPU može da pozove da se izvrše na GPU
        - Izvršenje N puta u paraleli od strane N različitih CUDA niti
        - Ne vraćaju vrednost • Svaka nit će izvršiti VecAdd()
    - Svaka nit ima jedinstveni thread ID pomoću koga joj se može pristupiti unutar kernela kroz ugrađenu threadIdx varijablu

```
// Kernel definition
__global__ void VecAdd(float* A, float* B, float* C) {
int i= threadIdx.x;
…
}

int main() {
…
// Kernel invocation with N threads
VecAdd<<<1, N>>>(A, B, C);
}
```

    - GPU stvara m blokova sa n niti (ukupan broj niti m*n) koje startuju kopiju iste funkcije KernelName<<>>(arg1, arg2, ...)
    - CPU može da nastavi procesiranje dok GPU startuje kernel. Bilo koji poziv kernel funkcije je asinhroni, eksplicitni synch je potreban za blokiranje
    - Kada se sve niti završe kernel poziv se vraća
      kernel1<<>>(...); // kernel startuje izvršenje, CPU nastavlja sa sledećom naredbom
      kernel2<<>>(...); // kernel2 se smešta u red, i startovaće posle završetka kernel1, CPU nastavlja
      cudaMemcpy(...); // CPU se blokira dok se memorija ne iskopira. Kopiranje memorije startuje nakon što se svi prethodni CUDA pozivi završe.
17. Kernel i hijerarhija niti
    - Kernel se izvršava u paraleli preko skupa paralelnih niti
    - Sve niti koje su generisane startovanjem kernela se kolektivno zovu rešetka
    - Niti su organizovane u blokove, a blokovi su organizovani u rešetke
    - Blok niti je skup konkurentno izvršavanih niti koje mogu da sarađuju između sebe putem barrier – sinhronizacije i deljene memorije
    - Rešetka je niz blokova niti koji izvršavaju isti kernel
        - Čitaju ulaze i upisuju rezultate u globalnu memoriju
        - Sinhronizuju se između zavisnih kernel poziva
18. Identifikacija niti i bloka (threadIdx, blockIdx), dimenzije rešetke i bloka (gridDim,blockDim). Indeksiranje sa više dimenzija.
    - Dimension
        - gridDim specificira broj blokova u rešetki
        - blockDim određuje broj niti u svakom bloku
    - Index
        - blockIdx daje indeks bloka u rešetki

    - threadIdx daje indeks niti unutar bloka
- threadIdx je 3-komponentni vektor
    - Thread index can be 1D, 2D, or 3D
    - Thread blocks as a result can be 1D, 2D, or 3D
- Kako da pronađemo vezu između thread id i threadIdx?
    - 1D: tid= threadIdx.x
    - 2D block of size (Dx, Dy): thread ID of a thread of index (x, y) is (x + yDx)
    - 3D block of size (Dx, Dy, Dz): thread ID of a thread of index (x, y, z) is (x + yDx+ zDxDy)
- Niti u bloku se "nalaze" na istom jezgru, maksimalno 1024 niti po bloku
- Blokovi niti su organizovani u 1D, 2D, ili 3D rešetke
    - Takođe se nazivaju niz kooperativnih niti
    - Grid dimension je data sa gridDim varijablom
- Identifikacija bloka unutar rešetke sa blockIdx varijablom
- Block dimension je data sa blockDim varijablom

Indeksiranje sa više dimenzija

Mnoge aplikacije koriste 2D ili 3D setove niti.

Ako želimo da koristimo 2D set niti, onda
blockDim.x, blockDim.y daje dimenzije blokova, a
threadIdx.x, threadIdx.y daje indekse niti

Da bismo startovali kernel koristićemo:

```
dim3 nblocks(2,3);                    // 2 blocks in x, 3 blocks in y
dim3 nthreads(16,4);                  // 16 threads in x, 4 threads in y
my_new_kernel<<<nblocks, nthreads>>>(d_x);
```

Ovde je *dim3* specijalan CUDA tip podatka sa 3 komponente .x,.y, i .z koje se inicijalizuju na 1.

Identifikator niti *tid* sada dobija oblik za 2D ili 3D setove niti:

tid= threadIdx.x+ threadIdx.y blockDim.x+ threadIdx.z* blockDim.x* blockDim.y;*
3D block of size (Dx, Dy, Dz): thread ID of a thread of index (x, y, z) is *(x + yDx+ zDxDy)*

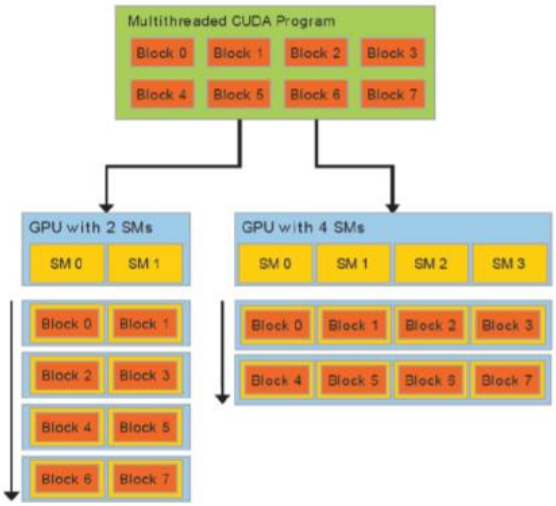
19. Sinhronizacija – sinhronizacija na barijeri, brava i semafori.
    - Sinhronizacija je neizostavna za koordinaciju paralelnih zadataka u vremenu izvršavanja. Po pravilu je povezana sa komunikacijom
    - Često se implementira kao tačka u programu koju zadatak ne sme da pređe pre nego što drugi zadaci dostignu logički ekvivalentnu tačku (barijera)
    - Sinhronizacija obično uključuje stanje čekanja bar jednog zadatka i zato produžava vreme izvršavanja paralelne aplikacije
    - Sinhronizacija na barijeri (barrier)
        - Obično uključuje sve zadatke
        - Svaki zadatak radi posao dok ne dođe do barijere, kada prekida posao i blokira se

        - Kada poslednji zadatak dosegne barijeru svi zadaci su sinhronizovani
            - Barijeru obično sledi kritična sekcija koja se mora sekvencijalno izvršiti
            - Zadaci se deblokiraju i nastavljaju svoj posao
    - Brava (lock)
        - Može učestvovati bilo koji broj zadataka
            - Samo jedan zadatak može istovremeno da koristi bravu i uđe u kritičnu sekciju
            - Prvi zadatak koji naiđe na slobodnu bravu je zaključava i obavlja posao, ostali moraju da čekaju
    - Semafori (semaphores)
        - Može učestvovati bilo koji broj zadataka
        - Određeni broj zadataka može istovremeno da koristi semafor i koristi resurs ili uđe u kritičnu sekciju
        - Blokirajući, binarni (brave) i brojački semafori

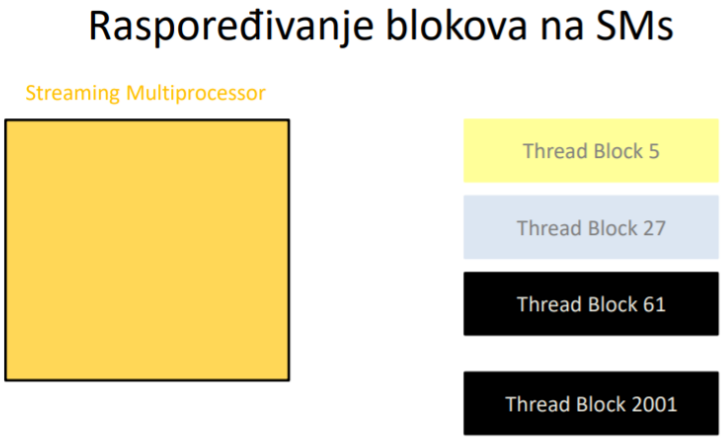
20. Sinhronizacija u CUDA jeziku: __syncthreads() i cudaDeviceSynchronize()
    - 1. __syncthreads() sinhronizuje niti unutar bloka
        - Mora da bude izvršena od strane svih niti u bloku
        - To je if naredba: ili sve niti u bloku izvršavaju putanju koja uključuje __syncthreads() ili ni jedna
    - 2. cudaDeviceSynchronize() sinhronizuje sve niti u rešetki
    - Za niti iz različitih rešetki, sistem omogučuje writes od kernela koji se dešavaju pre reads od sledeće “pokrenute” rešetke
    - 3. Atomic operacije predupređuju konflikte pridružene nitima koje konkurentno pristupaju varijabli
        - Atomic operacije su primenjene nad varijablama i u globalnoj i u deljenoj memoriji
        - Npr. float atomicAdd(float* addr, float amount)
    - Sve niti unutar vorpa izvršavaju se u lockstep-u. SM zapravo izvršava istu instrukciju u paraleli u isto vreme samo na različitim podacima. Niti su eksplicitno sinhronizovane
        - Lockstep sistemi su fault-tolerant sistemi koji izvršavaju isti set operacija u isto vreme u paraleli
    - __syncthreads() se koristi unutar kernela. Obezbeđuje barijeru za sinhronizaciju niti unutar bloka
        - stopira nit dok sve ostale niti ne stignu do te barijere lokacije
    - Koristi se da bi se izbegli RAW (read after write) / WAR / WAW hazardi kada se pristupa deljenoj ili globalnoj memoriji
    - Kada se piše uslovni kod veoma je važno da se omogući da su sve niti sposobne da stignu do __syncthreads(); U suprotnom dolazi do potpunog zastoja (deadlock) programa.
    - Dozvoljena je upotreba u uslovnim konstrukcijama (conditional constructs) samo ako je uslov - kondicional uniforman kroz ceo blok

- U glavnom programu main() možemo da sinhronizujemo sve na host-u i device-u do jedne tačke koda:
  cudaDeviceSynchronize(); poziv sa hosta koji je blokiran sve dok se GPU kerneli ne kompletiraju
- Ovo omogućava da se svi asinhroni (zovu se i ne-blokirajući) taskovi završe pre nego što se izvršenje koda nastavi
- cudaStreamSynchronize() čeka dok se svi kerneli u stream-u (niz operacija) ne završe
- Nema sinhronizacije između blokova

21. Nedeljive operacije (atomics)
    - Nedeljiva (atomic) operacija garantuje da samo jedna nit ima pristup delu memorije dok se operacija ne završi.
    - Naziv atomic dolazi od grčke reči atom što znači nedeljiv, a ovde se primenjuje kao neprekidna (uninterruptable) operacija
    - Različiti tipovi atomic instrukcija su
      - atomicAdd(), atomicSub(), atomicMin(), atomicMax(), atomicInc(), atomicDec(), atomicExch(), atomicCAS()
    - Atomic operacije
      - Na primer float atomicAdd(float* addr, float amount)
    - CUDA obezbeđuje atomic operacije i u globalnom adresnom prostoru i za deljeni adresni prostor bloka.
    - An atomic function
      - Must guarantee that an operation can complete without interference from any other thread
      - Does not provide any guarantee of ordering or provide any synchronisation
    - Atomic operacije usporavaju izvršenje programa i uveliko zavise od generacije arhitekture
22. Skalabilnost GPU arhitekture i raspoređivanje blokova niti
    - Multithreaded program se deli na blokove niti koji se izvršavaju nezavisno jedni od drugih
    - GPU sa više multiprocesora će automatski izvršiti program za kraće veme od GPU-a sa nekoliko multiprocesora



➔ Rasporedjivanje blokova niti



- HW raspoređuje blokove niti na SM multiprocesore koji su na raspolaganju
  Nema garancije o redosledu između blokova niti
  HW će rasporediti blokove niti čim se prethodni blok niti završi

- Broj niti koje mogu da se simultano prate i raspoređuju je ograničen
  - Zahtevaju se resursi za jedan SM da održavaju indekse blokova i niti i njihove statuse izvršavanja
- Do 2048 niti može da se dodeli svakom SM za poslednje generacije CUDA devices
  - Na primer: 8 blocks of 256 threads, or 4 blocks of 512 threads
- Ako pretpostavimo da CUDA device ima 28 SMs
  - Svaki SM može da “smesti” do 2048 niti
  - Device može imati do 57344 niti koje se simultano nalaze na GPU device za izvršenje

23. CUDA memorije (registri, lokalna memorija, deljena memorija, globalna i konstantana memorija) – karakteristike, prava pristupa, način deklarisanja promenljivih
    - Registri: najbrži oblik memorije na multiprocesoru. Mogu im pristupiti samo niti. Imaju životni vek kao niti.
      - “debele” niti, koriste dosta registara, i

    - “tanke” niti zahtevaju samo nekoliko!
- Lokalna memorija: Smeštena u globalnoj memoriji i može biti 150x sporija od registara ili deljene memorije. Može joj pristupiti samo nit. Ima trajanje kao nit.
- Deljena memorija: može biti brza kao registar kada ne postoje bank konflikti kada se čita sa iste adrese. Može joj pristupiti bilo koja nit bloka koji ju je kreirao. Ima životni vek bloka.
    - ~5ns of latency
    - Maximum size of ~48KB (varies per GPU)
- Globalna memorija: Potencijalno 150x sporija od registara ili deljene memorije. Treba voditi računa o uncoalesced čitanju i upisu. Može joj se pristupiti ili sa host-a ili sa device-a. Traje dok traje aplikacija
    - GPUs have .5 - 24GB of global memory, with most now having ~2GB.
    - Global memory latency is ~300ns on Kepler and ~600ns on Fermi